

机器学习导论-神经网络

Introduction to Machine Learning-Neural Networks

李文斌

<https://cs.nju.edu.cn/liwenbin>

liwenbin@nju.edu.cn

2024年04月09日

大纲

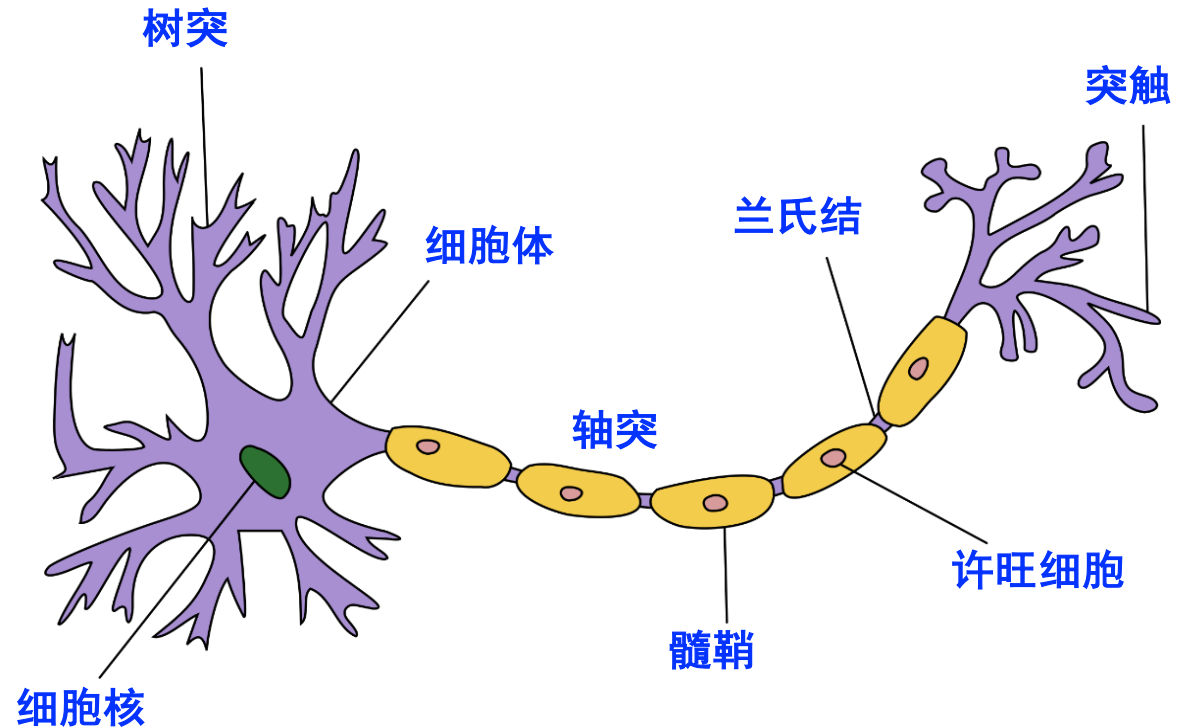
- **脑和神经元**
- **激活函数**
- **感知机学习**
- **多层感知机**
- **反向传播算法**

生物学基础

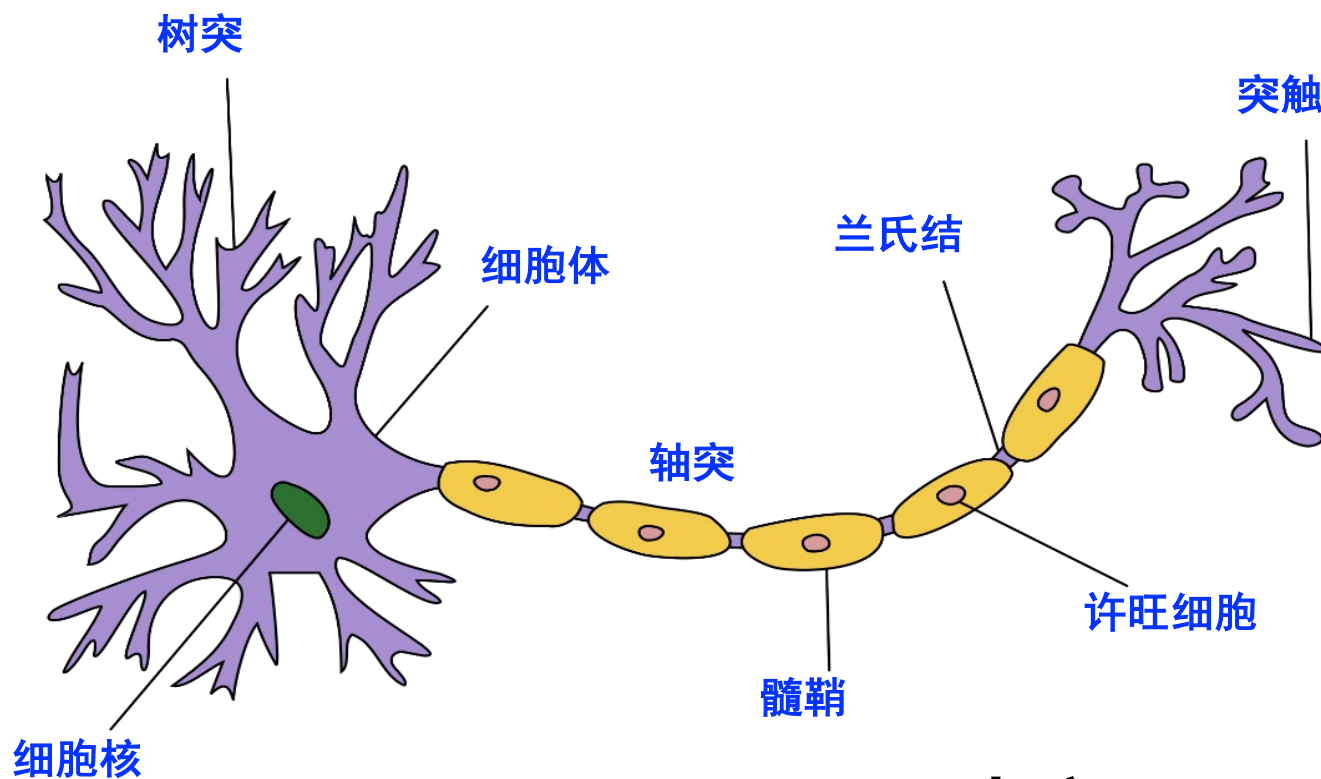
树突：输入

轴突：输出

- ✓ 神经元内化学物质调节内部电位
- ✓ 跨膜电位达到一个阈值时，则激活或放电
- ✓ (固定)时间和强度的脉冲传递给轴突
- ✓ 轴突像树枝状，连接到突触



生物学基础



□ 人脑

- ✓ 分布/并行计算模型
- ✓ 调整节点间的连接关系

- ✓ 10^{11} 个神经元 (100 billion)
- ✓ 每个神经元处理速度为 $10^{-3}s$
- ✓ 识别人脸, 需要约100个神经元计算

赫布理论(Hebbian Theory)

□ 唐纳德·赫布 (1904-1985) 于1949年提出

- 加拿大生理心理学家
- 连接强度调整量与输入输出的乘积成正比，显然经常出现的模式将增强神经元之间的连接
- 与巴普洛夫的 ‘条件反射’ 一致
- 又称为 “长程增强机制” (LTP, Long Term Potentiation) 或 “神经可塑” (Neural Plasticity)

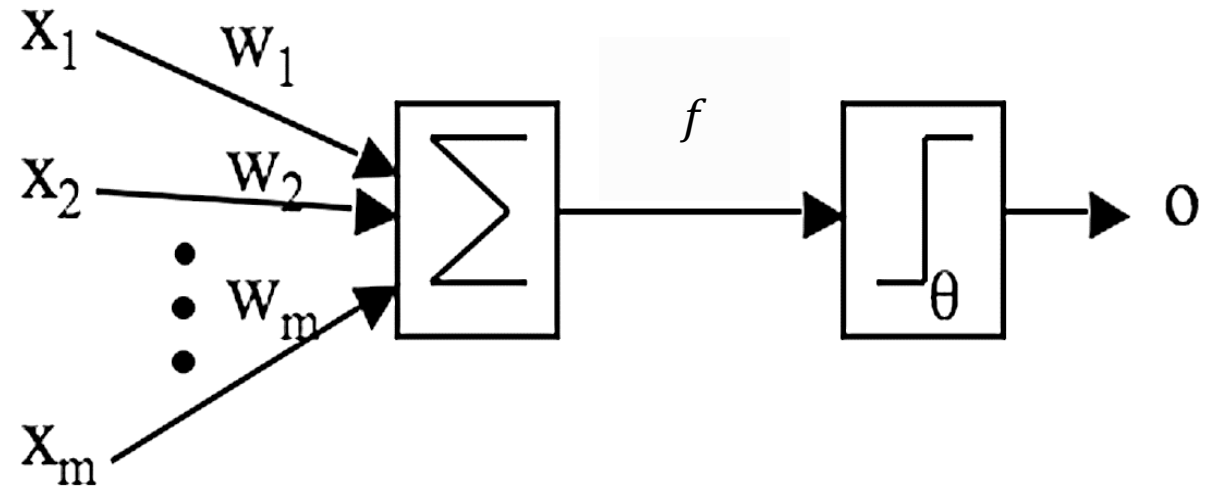
早期发展历史

- ❑ 1943年，心理学家W. McCulloch和数理逻辑学家W. Pitts首次提出神经元的数学模型MP模型
- ❑ 1948年，冯·诺依曼提出相互再生自动机网络结构
- ❑ 1950s，F. Rosenblatt提出感知机模型
- ❑ 1960s，Widrow提出非线性多层自适应网络
- ❑ 1969年，Minsky出版《感知机》一书
- ❑ 1982年和1984年，物理学家Hopfield在美国科学院院刊上发表ANN文章
- ❑ 2006年，Hinton发表深度信念网络→深度学习

。 。 。

MP神经元基本结构

- 输入 $X = [x_1, x_2, x_3, \dots]$
- 权值 $W = [w_1, w_2, w_3, \dots]$
- **激活函数** $f(net) = f(\sum(w_i * x_i))$
- 偏置单元 (bias unit) x_0 , 其对应权值为 w_0



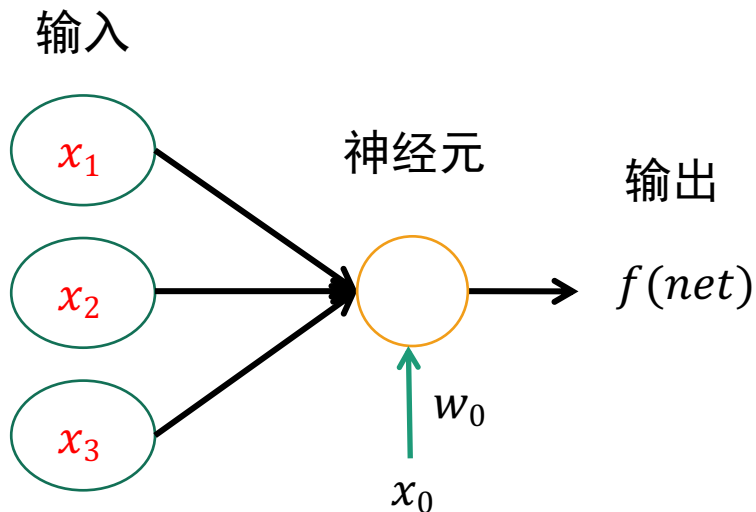
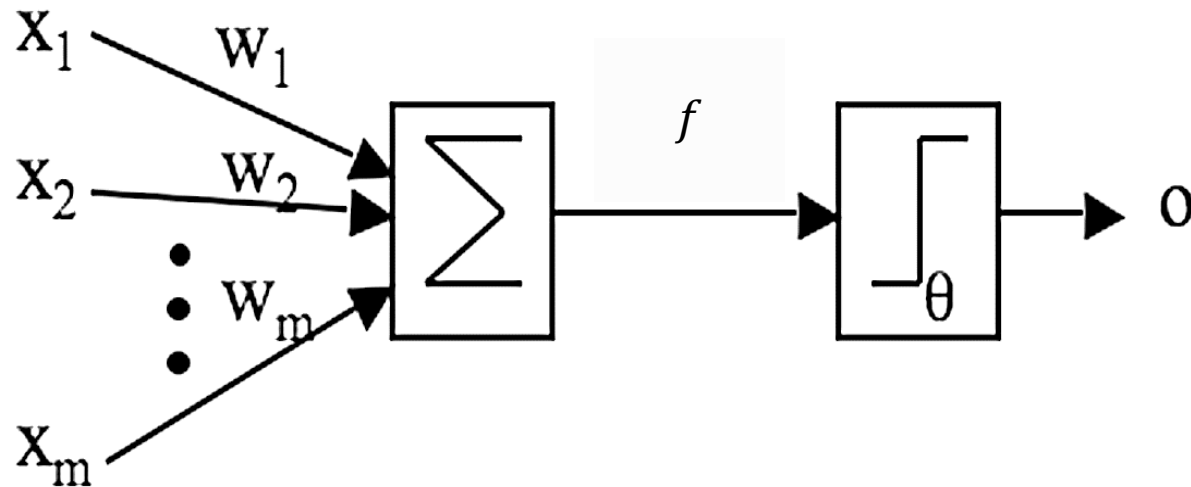
MP神经元基本结构

□ 输入 $X = [x_1, x_2, x_3, \dots]$

□ 权值 $W = [w_1, w_2, w_3, \dots]$

□ **激活函数** $f(net) = f(\sum(w_i * x_i))$

□ 偏置单元 (bias unit) x_0 , 其对应权值为 w_0



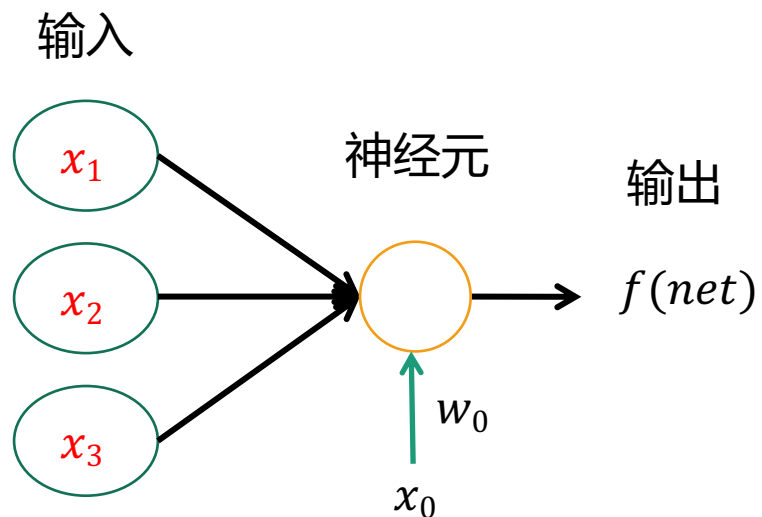
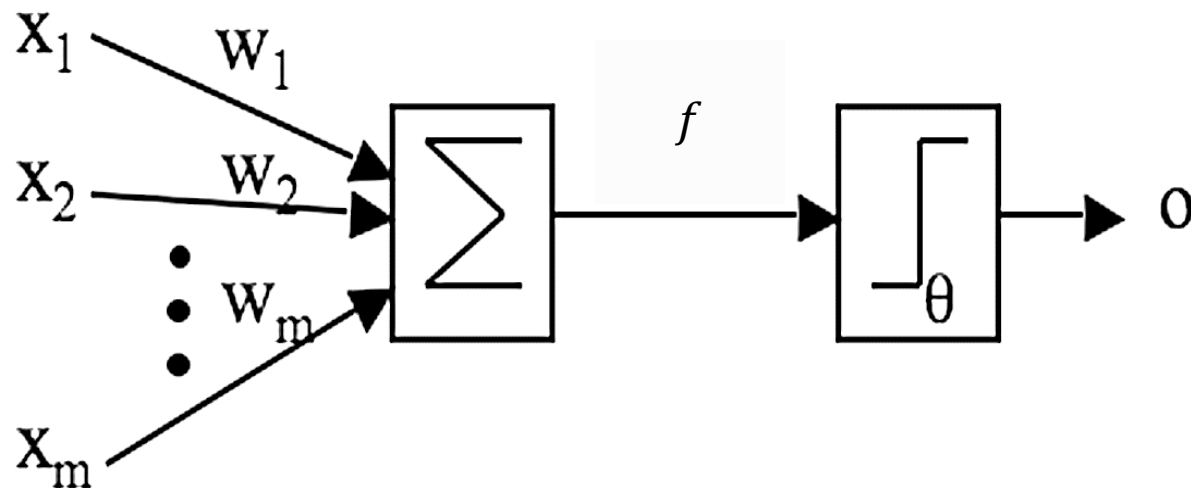
MP神经元基本结构

□ 输入 $X = [x_1, x_2, x_3, \dots]$

□ 权值 $W = [w_1, w_2, w_3, \dots]$

□ 激活函数 $f(net) = f(\sum(w_i * x_i))$

□ 偏置单元 (bias unit) x_0 , 其对应权值为 w_0

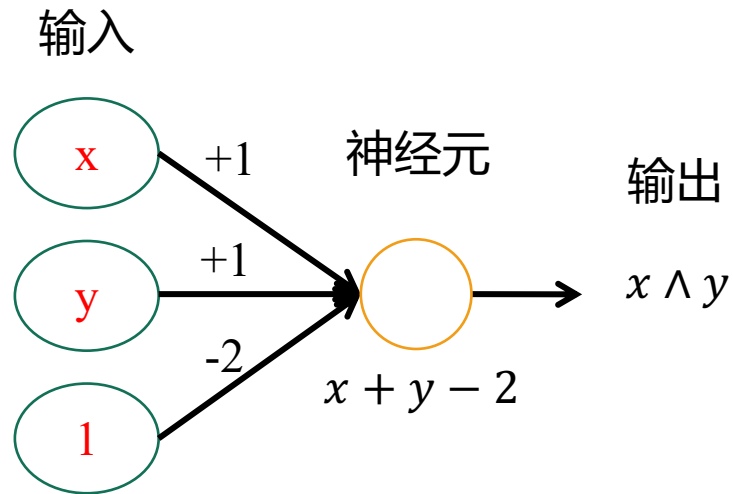


- ✓ 一组输入加权 w_i 相当于突触
- ✓ 一个加法器把输入信号相加 (与收集电荷的细胞膜等价)
- ✓ 一个激活函数 (最初是一个阈值函数) 决定细胞对于当前的输入是否激活 ("放电")

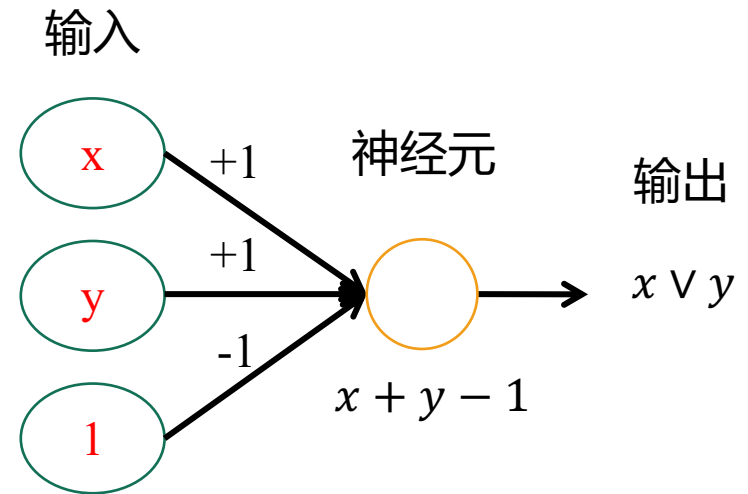
MP神经元模型

$$h = \sum_{i=1}^m w_i x_i$$

$$o = g(h) = \begin{cases} 1, & \text{if } h > 0 \\ 0, & \text{if } h \leq 0 \end{cases}$$



逻辑与的神经元模型
(阈值为0, 大于等于0输出1)



逻辑或的神经元模型
(阈值为0, 小于0输出0)

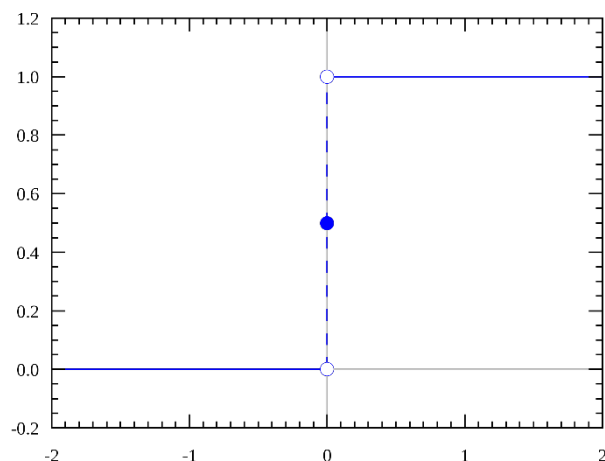
MP神经元的局限

- **输入方面**：线性求和 → 非线性求和
- **输出方面**：单一输出值 → 脉冲序列
- **更新机制**：时钟同步更新 → 随机异步更新
- **权值的物理（生理）意义**
 - ✓ 两类神经元连接（兴奋性连接，权值为正；抑制性连接，权值为负）
 - ✓ 但不存在由正到负、或者由负到正的连接

大纲

- 脑和神经元
- 激活函数
- 感知机学习
- 多层感知机
- 反向传播算法

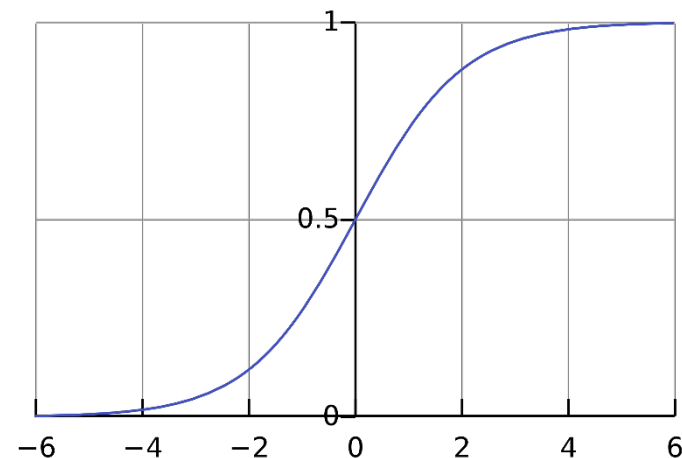
激活函数



单位阶跃函数
Heaviside step function

$$f(x) := \begin{cases} 1, & \text{if } x \geq 0 \\ 0, & \text{if } x < 0 \end{cases}$$

- 不连续函数
- 对变化敏感
- $x = 0$, 不可微
- 一般适合单层感知机

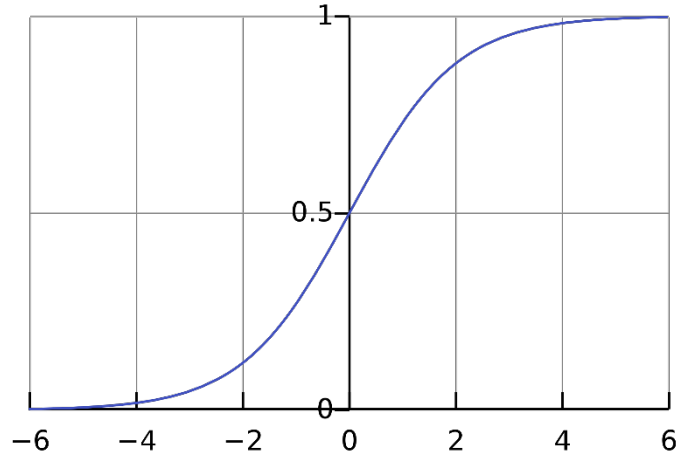


Sigmoid function

$$f(x) := \frac{1}{1 + e^{-x}}$$

- 连续、光滑、严格单调
- 函数范围在(0,1)之间
- S形曲线, 非线性函数
- 导数为其本身的函数

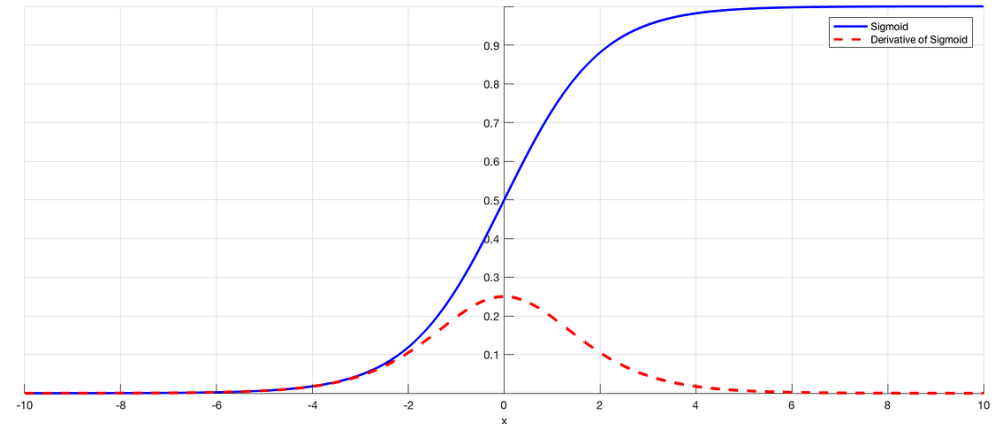
激活函数



Sigmoid function

$$f(x) := \frac{1}{1 + e^{-x}}$$

- 连续、光滑、严格单调
- 函数范围在(0,1)之间
- S形曲线，非线性函数
- 导数为其本身的函数

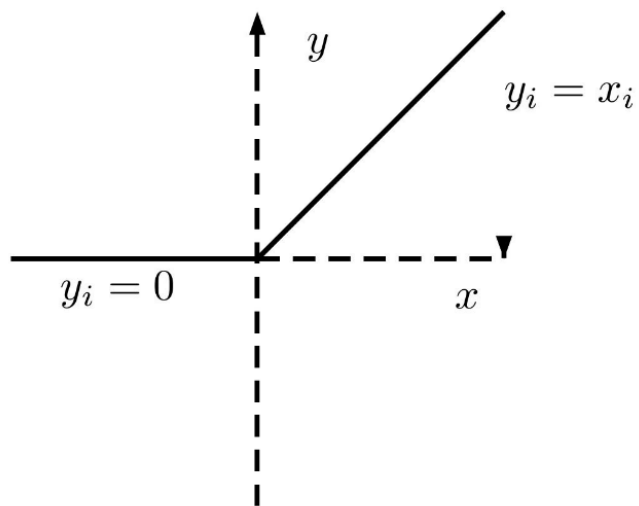


导数: $f'(x) = f(x)(1 - f(x))$

- 饱和型激励函数
- 导数始终小于1，在0周围变化
- 容易造成梯度消失问题
- 指数计算代价大

激活函数

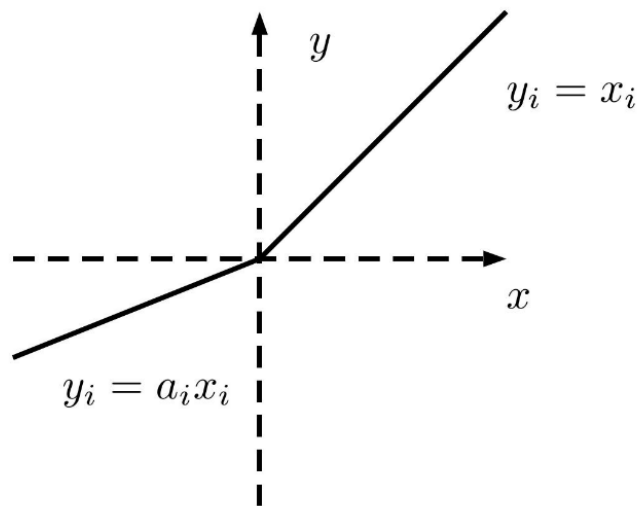
- **饱和激活函数**: Sigmoid, tanh, ...
- **非饱和激活函数**: ReLU, ELU, Leaky ReLU, PReLU, RReLU, ...



ReLU

$$f(x) := \max(0, x)$$

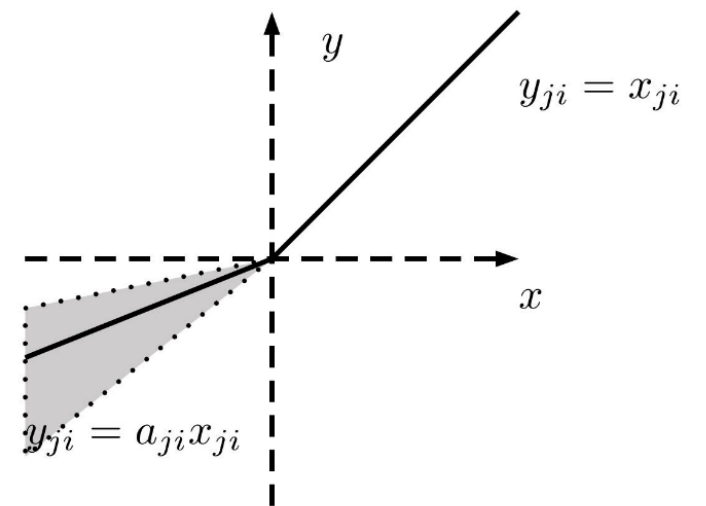
$$f'(x) := \begin{cases} 1, & \text{if } x > 0 \\ 0, & \text{if } x < 0 \end{cases}$$



Leaky ReLU/PReLU

$$f(x) := \begin{cases} x, & \text{if } x \geq 0 \\ a \cdot x, & \text{if } x < 0 \end{cases}$$

$$f'(x) := \begin{cases} 1, & \text{if } x \geq 0 \\ a, & \text{if } x < 0 \end{cases}$$



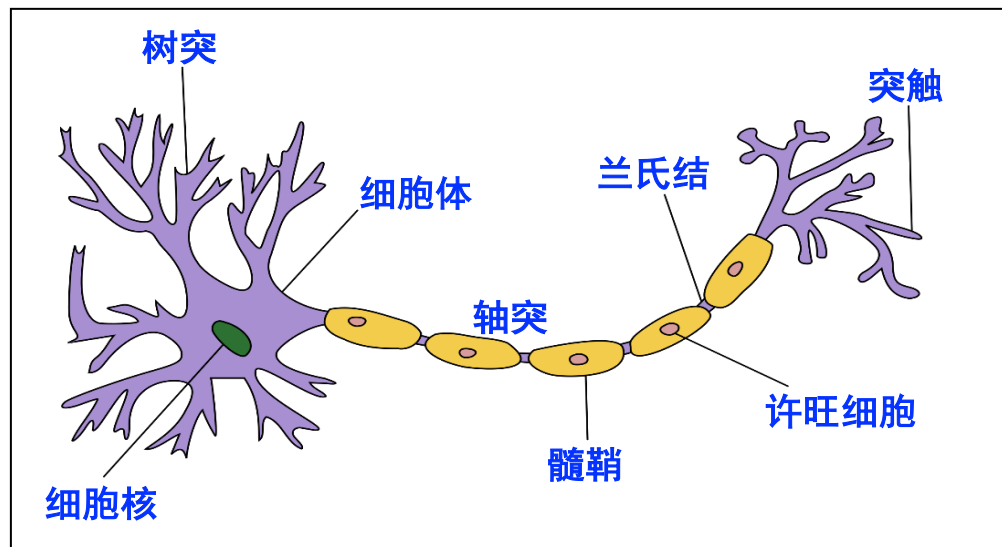
Randomized Leaky ReLU

$$f(a, x) := \begin{cases} x, & \text{if } x \geq 0 \\ a \cdot x, & \text{if } x < 0 \end{cases}$$

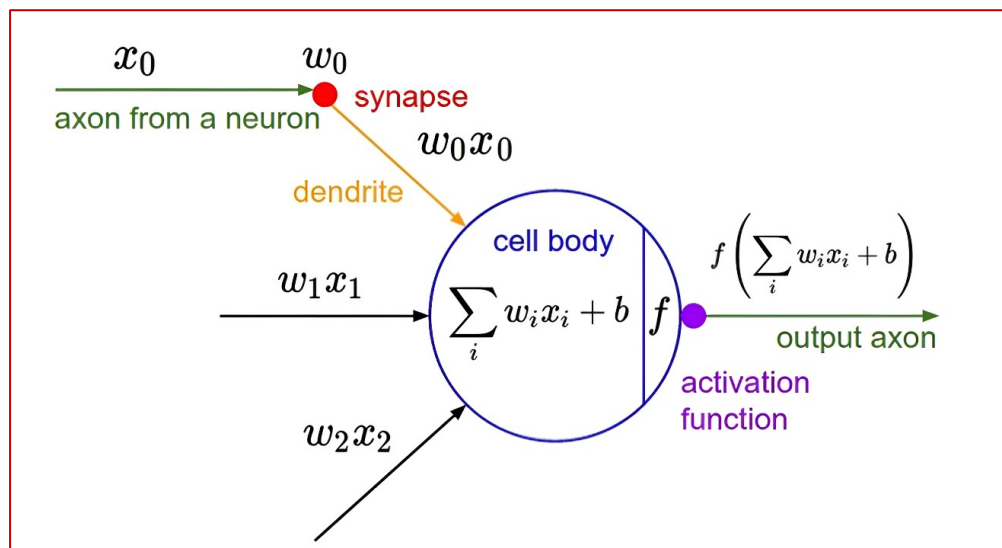
训练阶段 a 从一个均匀分布中随机采样

生物神经元→计算神经元

生物神经元



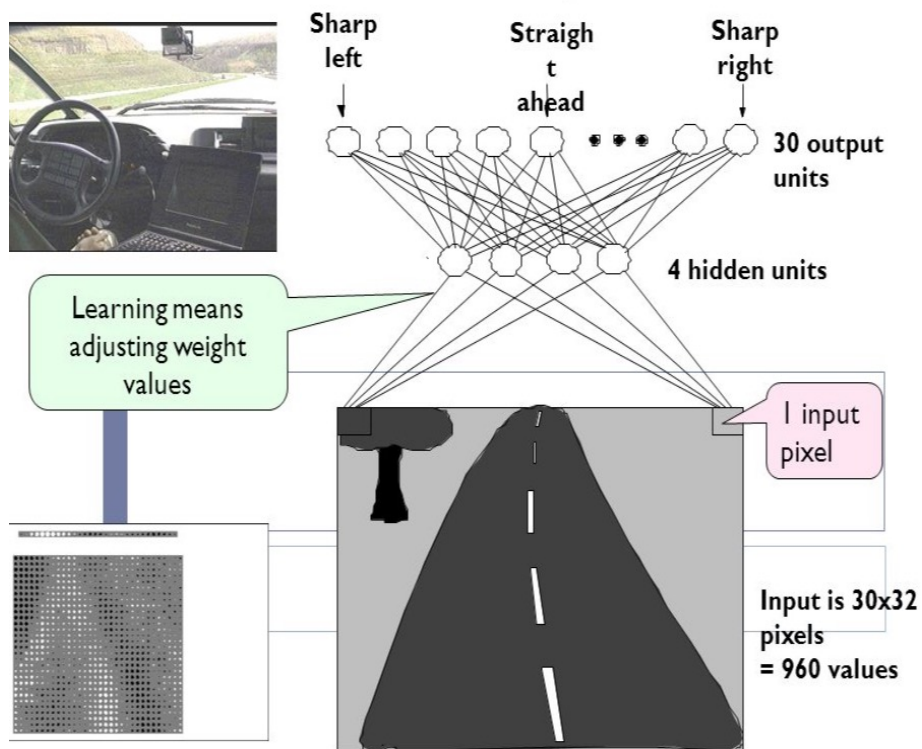
计算神经元



神经元→神经网络表示

□ ALVINN : **A**utonomous **L**and **V**ehicle **I**n a **N**eural **N**etwork

Neural Net example: ALVINN



□ 1989年, CMU研发

□ 输入: 30*32的像素

□ 4个隐藏节点

□ 输出: 30个驾驶动作(急剧左转、急剧右转, 正前方行进.....)

大纲

- 脑和神经元
- 激活函数
- 感知机学习
- 多层感知机
- 反向传播算法

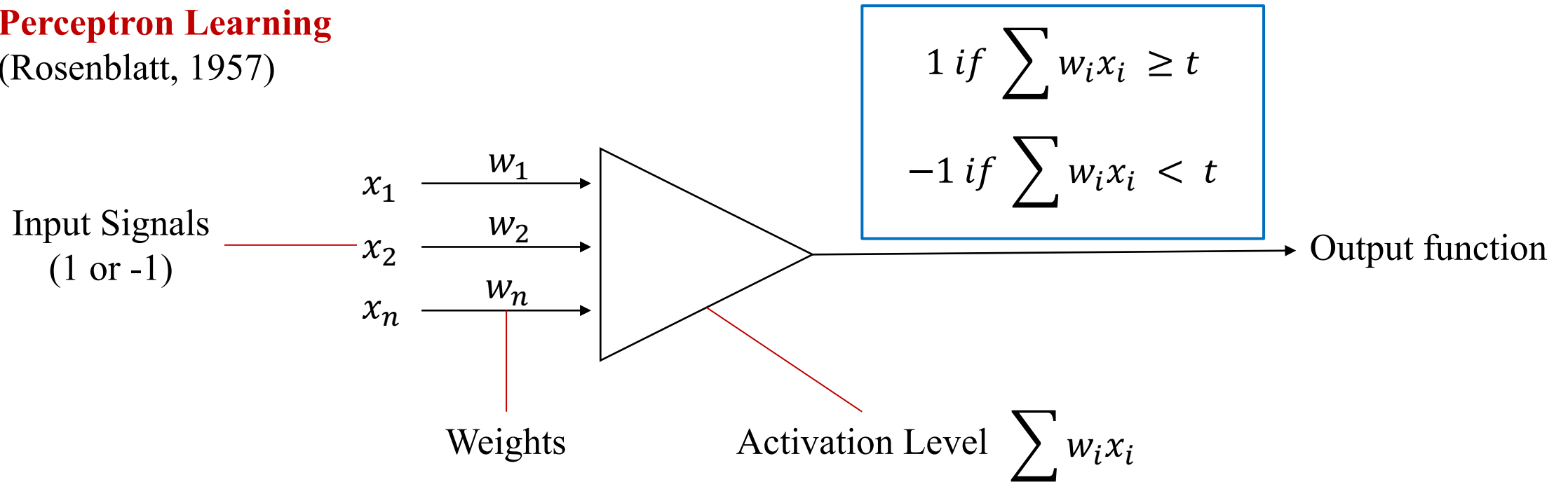
感知机和感知机学习

- Frank Rosenblatt, 1957年, Cornell航空实验室 (Cornell Aeronautical Laboratory)
- 最简单形式的前馈式人工神经网络
- 一种二元线性分类器, 使用特征向量作为输入, 把矩阵上的输入 x (实数值向量) 映射到输出值 $f(x)$ 上 (一个二元的值)

感知机结构

Perceptron Learning

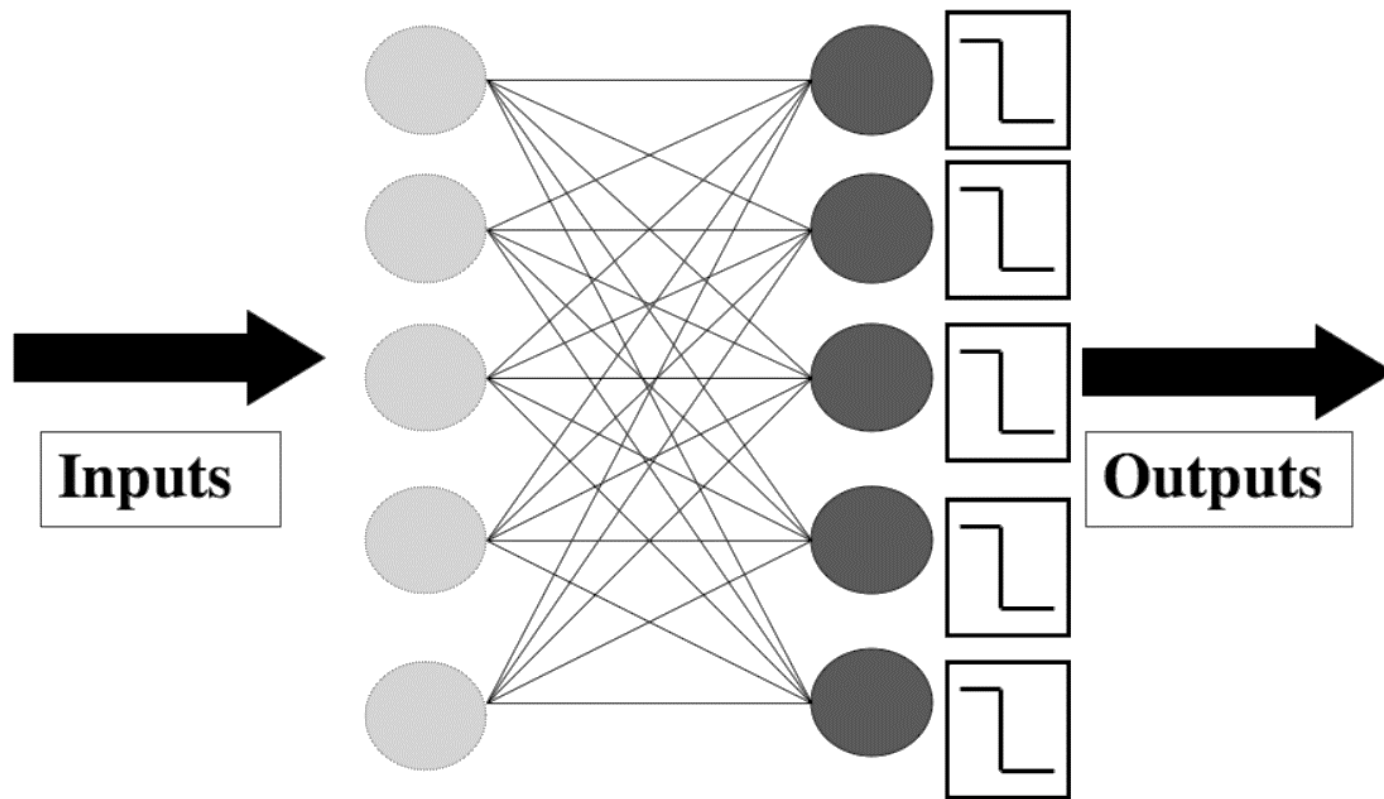
(Rosenblatt, 1957)



Learning Function

$$\Delta W_i = c(d - \text{sign}\left(\sum w_i x_i\right))x_i$$

感知机结构



□ 非线性前馈网络

- 同层内无互连
- 不同层间无反馈
- 由下层向上层传递
- 输入输出均为离散值
- 由阈值函数决定其输出

有监督的学习机制

□ η 是常数，表示学习率

□ y_j 是期望的输出，取值为1或-1

$$w_{ij} \leftarrow w_{ij} + \eta(y_j - t_j) \cdot x_i$$

□ sign 是感知机的输出，取值为1或-1

$$\Delta W_i = \eta(y_j - \text{sign}\left(\sum w_i x_i\right))x_i$$

□ 期望输出和实际输出相同，不改变权值

□ 实际输出为-1，期望输出为+1，则增加 $2\eta x_i$

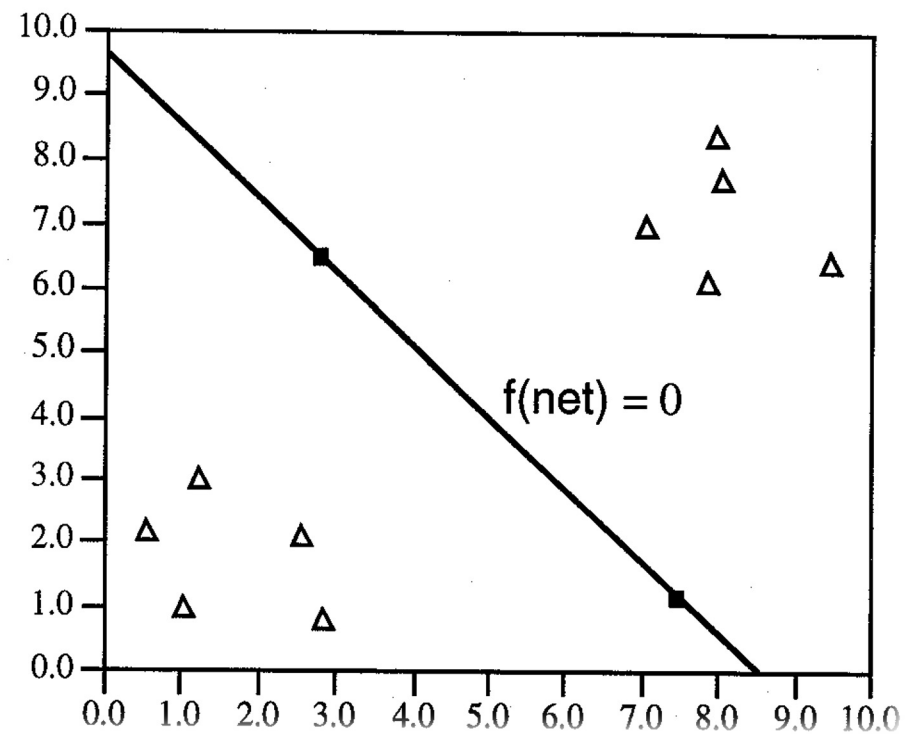
□ 实际输出为+1，期望输出为-1，则减少 $2\eta x_i$

感知机的学习算法

1. 权值初始化
2. 输入样本对
3. 计算输出
4. 根据感知机学习规则调整权重
5. 返回步骤2，输入下一对样本，直至对所有样本的实际输出与期望输出相等

例子

x_1	x_2	Output
1.0	1.0	1
9.4	6.4	-1
2.5	2.1	1
8.0	7.7	-1
0.5	2.2	1
7.9	8.4	-1
7.0	7.0	-1
2.8	0.8	1
1.2	3.0	1
7.8	6.1	-1



例子

x_1	x_2	Output
1.0	1.0	1
9.4	6.4	-1
2.5	2.1	1
8.0	7.7	-1
0.5	2.2	1
7.9	8.4	-1
7.0	7.0	-1
2.8	0.8	1
1.2	3.0	1
7.8	6.1	-1

$$f(net) = f(w_0 * x_0 + w_1 * x_1 + w_2 * x_2)$$

x_0 是偏置单元，通常取1

- 假设初始权值为[-0.6, 0.75, 0.5]

- 对第一行数据来说

$$\begin{aligned} f_1 &= f(-0.6 * 1 + 0.75 * 1 + 0.5 * 1) \\ &= f(0.65) = 1 \end{aligned}$$

- 与期望值一样，所以 W 向量不变， $W_1 = W_0$

例子

x_1	x_2	Output
1.0	1.0	1
9.4	6.4	-1
2.5	2.1	1
8.0	7.7	-1
0.5	2.2	1
7.9	8.4	-1
7.0	7.0	-1
2.8	0.8	1
1.2	3.0	1
7.8	6.1	-1

$$f(net) = f(w_0 * x_0 + w_1 * x_1 + w_2 * x_2)$$

x_0 是偏置单元, 通常取1

- 假设初始权值为 $[-0.6, 0.75, 0.5]$
- 对第一行数据来说
$$f_1 = f(-0.6 * 1 + 0.75 * 1 + 0.5 * 1)$$
$$= f(0.65) = 1$$
- 与期望值一样, 所以 W 向量不变, $W_1 = W_0$



- $$f_2 = f(-0.6 * 1 + 0.75 * 9.4 + 0.5 * 6.4)$$
$$= f(9.65) = 1$$
- 期望为-1, 所以 $W_2 = W_1 + 0.2 * (-2)X_2$
$$W_2 = [-0.6, 0.75, 0.5] - 0.4 * [1, 9.4, 6.4]$$
$$= [-1.00, -3.01, -2.06]$$

例子

$$f(\text{net}) = f(w_0 * x_0 + w_1 * x_1 + w_2 * x_2)$$

x_0 是偏置单元，通常取1

x_1	x_2	Output
1.0	1.0	1
9.4	6.4	-1
2.5	2.1	1
8.0	7.7	-1
0.5	2.2	1
7.9	8.4	-1
7.0	7.0	-1
2.8	0.8	1
1.2	3.0	1
7.8	6.1	-1

- 假设初始权值为 $[-0.6, 0.75, 0.5]$
- 对第一行数据来说
$$f_1 = f(-0.6 * 1 + 0.75 * 1 + 0.5 * 1)$$
$$= f(0.65) = 1$$
- 与期望值一样，所以 W 向量不变， $W_1 = W_0$

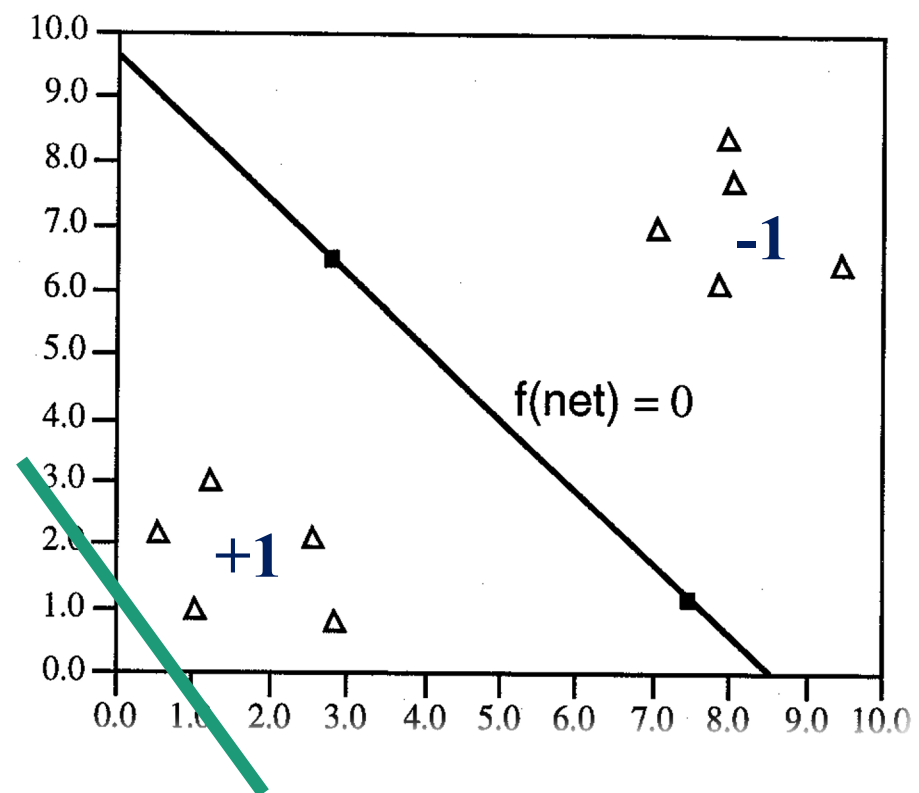


- $$f_2 = f(-0.6 * 1 + 0.75 * 9.4 + 0.5 * 6.4) = f(9.65)$$
- $$= 1$$
- 期望为-1，所以 $W_2 = W_1 + 0.2 * (-2)X_2$
$$W_2 = [-0.6, 0.75, 0.5] - 0.4 * [1, 9.4, 6.4]$$
$$= [-1.00, -3.01, -2.06]$$

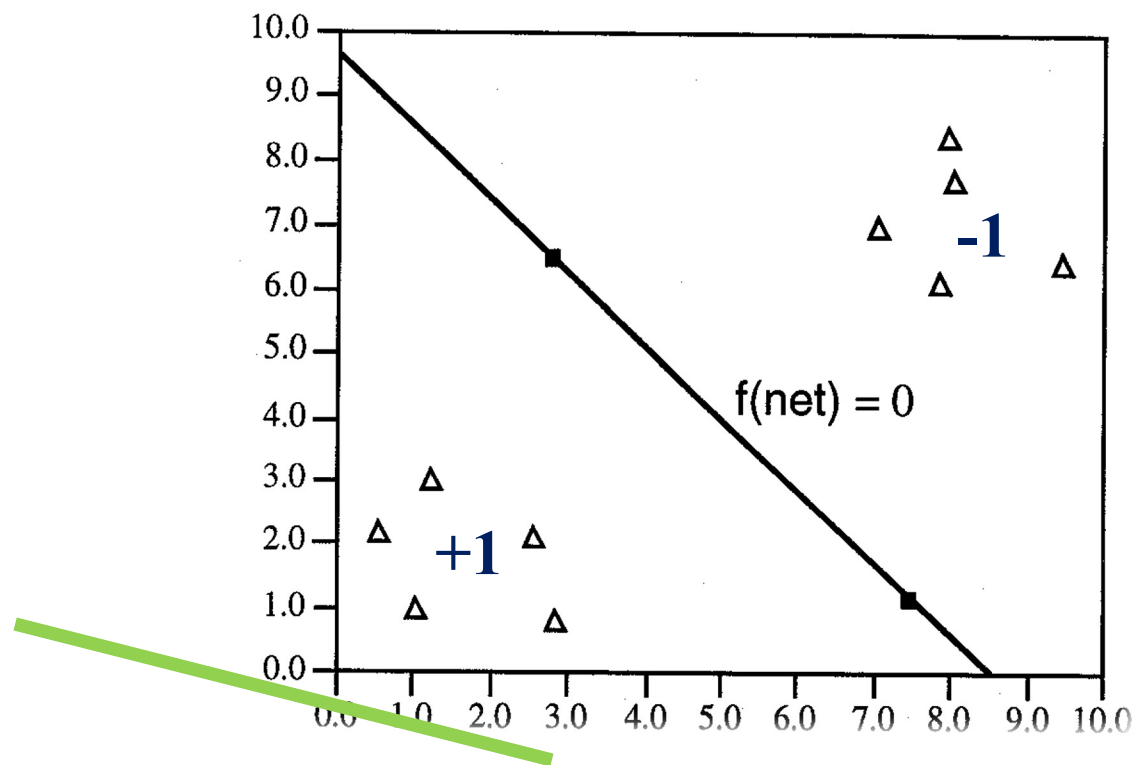


- $$f_3 = f(-1 * 1 - 3.01 * 2.5 - 2.06 * 2.1)$$
- $$= f(-12.84) = -1$$
- $$W_3 = W_2 + 0.2 * 2 * X_3 = [-0.60, -2.01, -1.22]$$
-
- 最终结果为 $W=[10.9, -1.3, -1.1]$

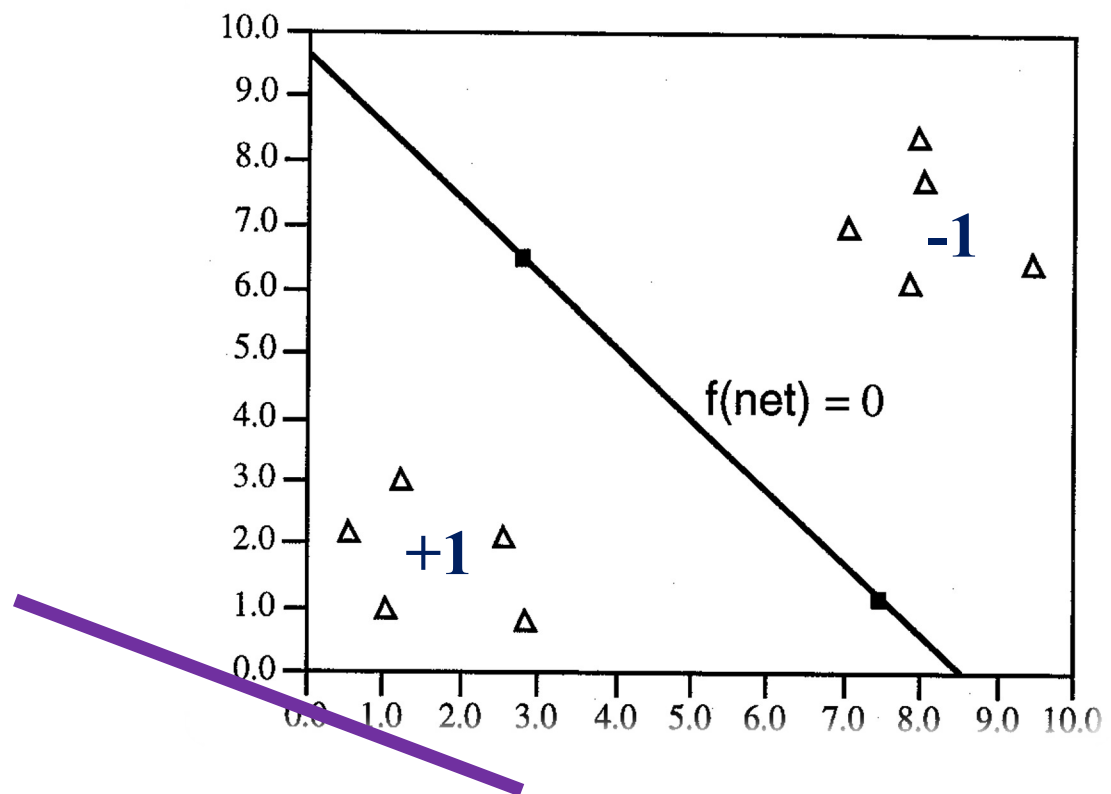
图示



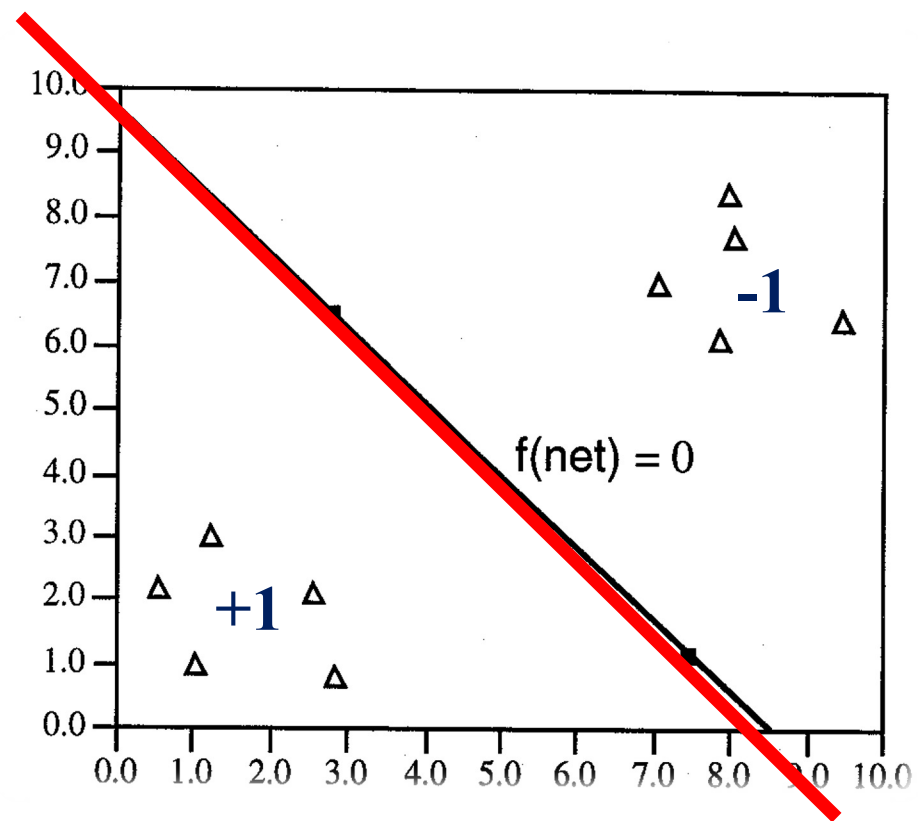
图示



图示

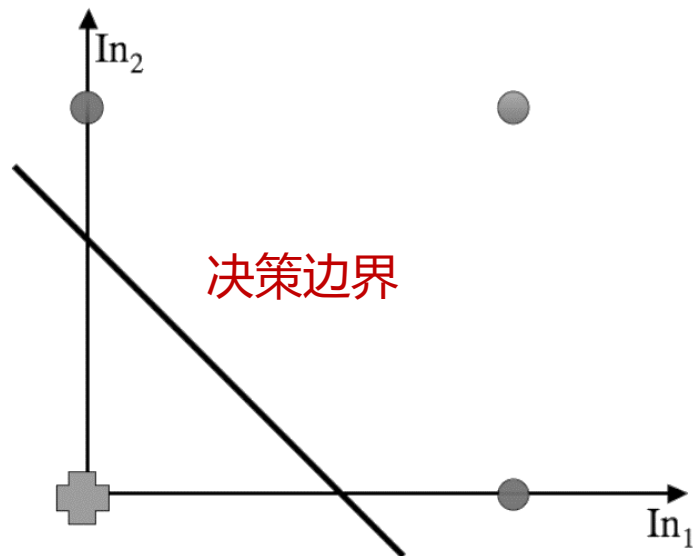


图示



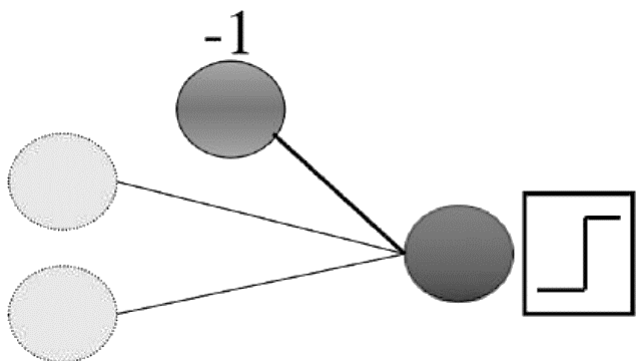
OR函数

In ₁	In ₂	t
0	0	0
0	1	1
1	0	1
1	1	1



$$w_{ij} \leftarrow w_{ij} - \eta(y_j - t_j) \cdot x_i$$

例题中, 令 $\eta = 0.25$



$$w_0 = -0.05,$$

$$w_1 = -0.02,$$

$$w_2 = 0.27,$$

课堂练习



$$w_0 = ?$$

$$w_1 = ?$$

$$w_2 = ?$$

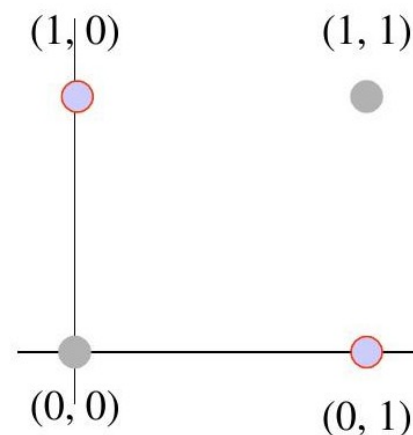
感知机学习缺点

- 感知机属于单层神经网络，不能解决一类**非线性可分**的问题
- 典型的例子就是**异或 xor**

表 1. 异或真值表

输入1	输入2	输出
0	0	0
0	1	1
1	0	1
1	1	0

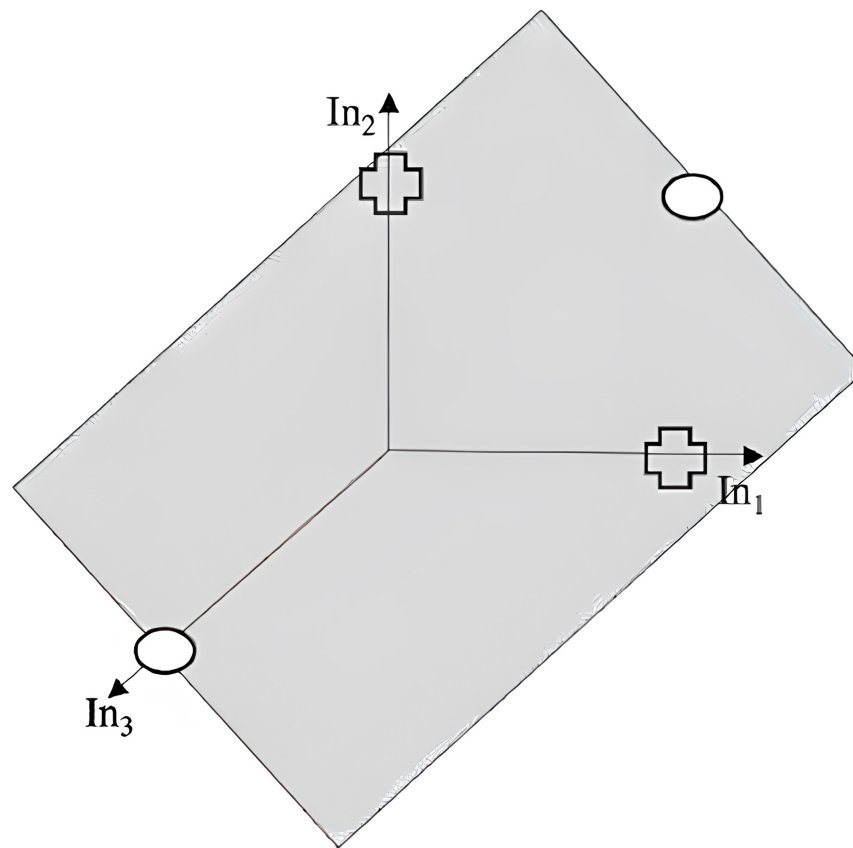
图 1. 异或问题



在二维空间中没有可分离点集 $\{(0, 0), (1, 1)\}$ 和 $\{(0, 1), (1, 0)\}$ 的直线

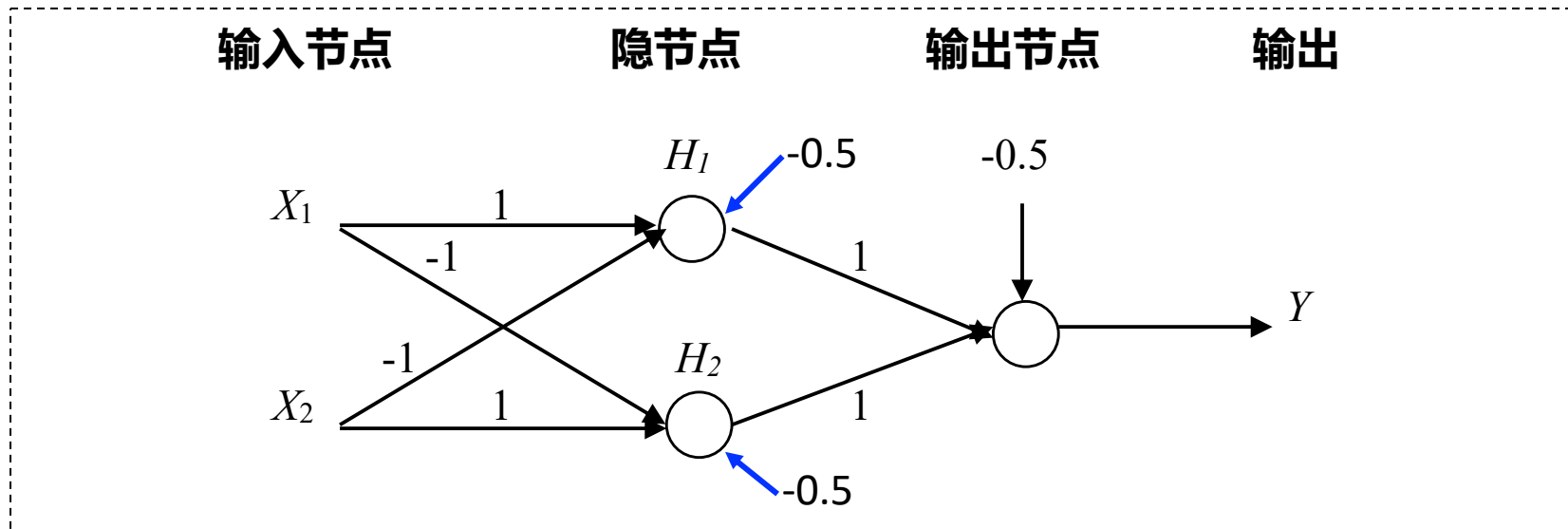
用线性分类器处理异或

In ₁	In ₂	In ₃	Output
0	0	1	1
0	1	0	0
1	0	0	0
1	1	0	1



利用核技巧（SVM一讲介绍），将数据投影到合适的高维空间

用多层网络处理异或



□ 权重向量(1,-1)、(-1,1)

输入		隐节点输出		输出节点	$X_1 \text{ xor } X_2$
X_1	X_2	H_1	H_2		
0	0	0	0	$-0.5 \rightarrow 0$	0
0	1	$-1 \rightarrow 0$	1	$0.5 \rightarrow 1$	1
1	0	1	$-1 \rightarrow 0$	$0.5 \rightarrow 1$	1
1	1	0	0	$-0.5 \rightarrow 0$	0

感知机学习的不足

□ 感知机学习一定可以收敛吗？

- ✓ 前提是训练样例必须是线性可分的！！

□ 如果训练样例不是线性可分的，怎么办？

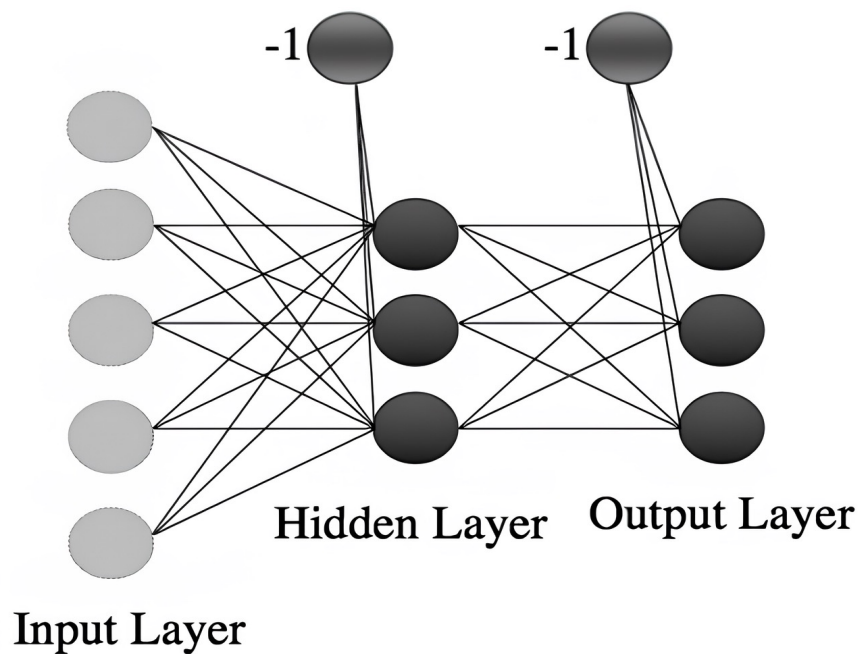
- ✓ 只能去找一个学习方法，去收敛到目标概念的最佳近似

感知机学习方法只适用在单层网络！

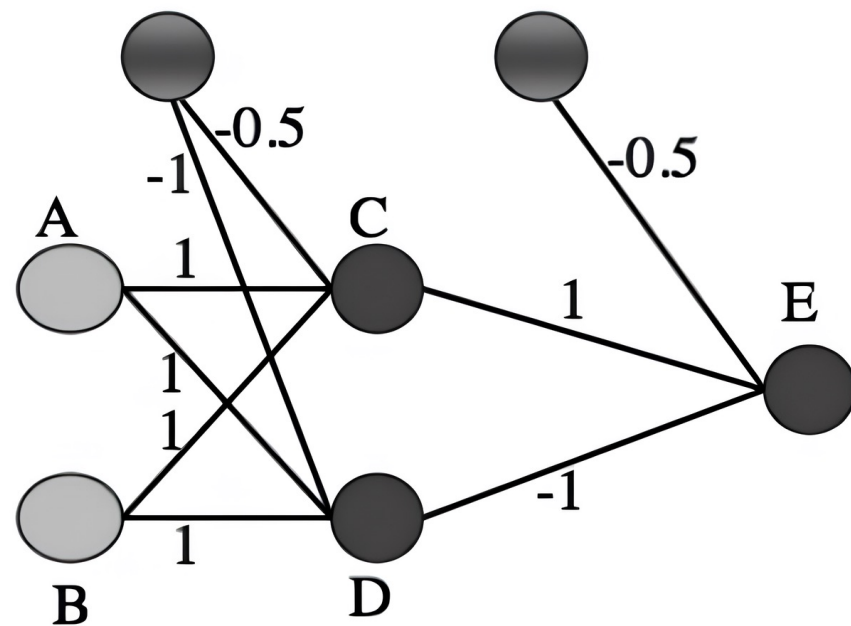
大纲

- 脑和神经元
- 激活函数
- 感知机学习
- 多层感知机
- 反向传播算法

多层感知机

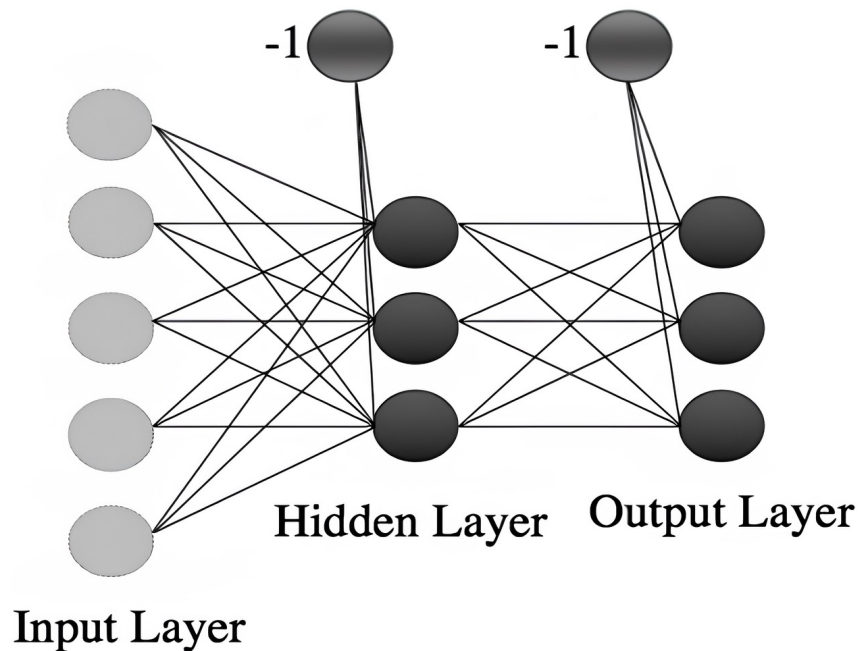


多层感知机网络



解决xor的多层感知机网络权值

多层感知机

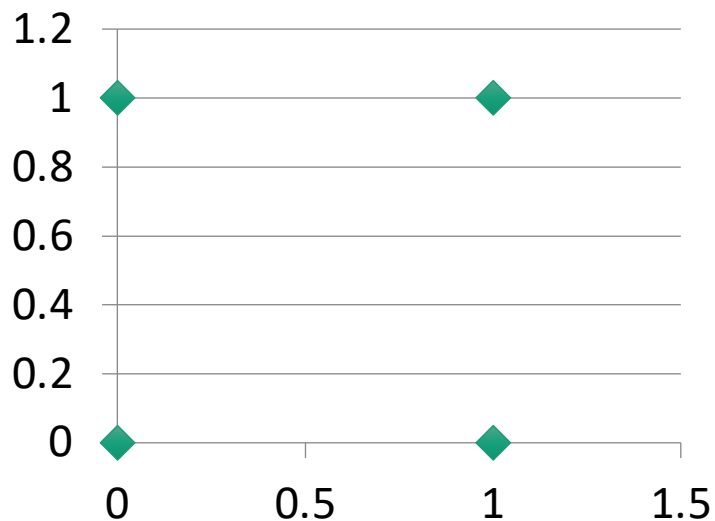


Q1: 为什么需要隐藏层?

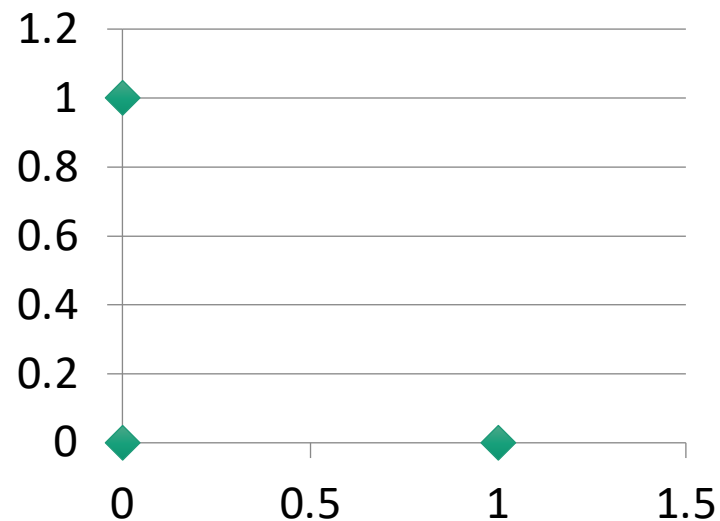
Q2: 如何计算隐藏层的权值?

多层感知机网络

多层感知机的隐层



单层神经网络的特征空间



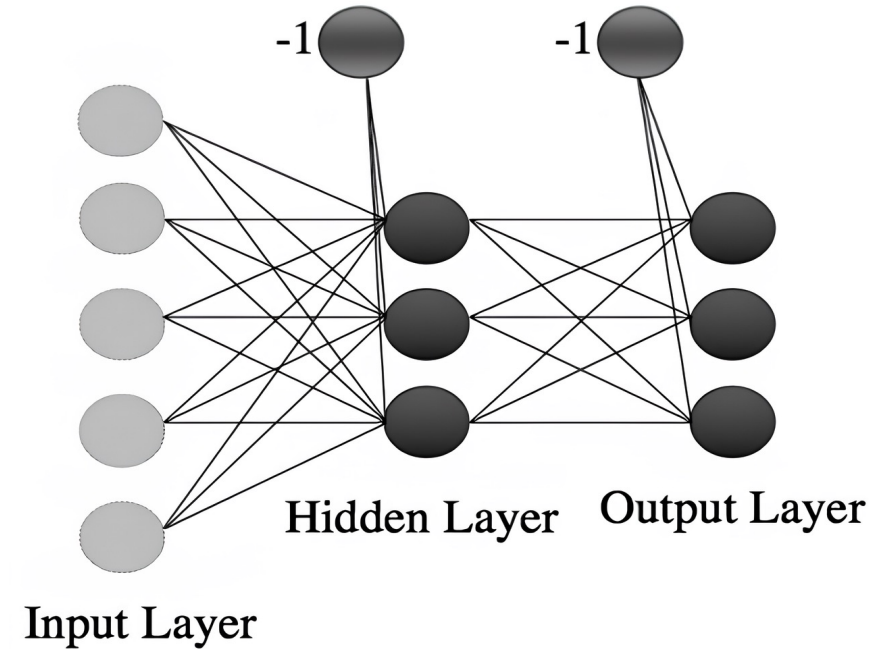
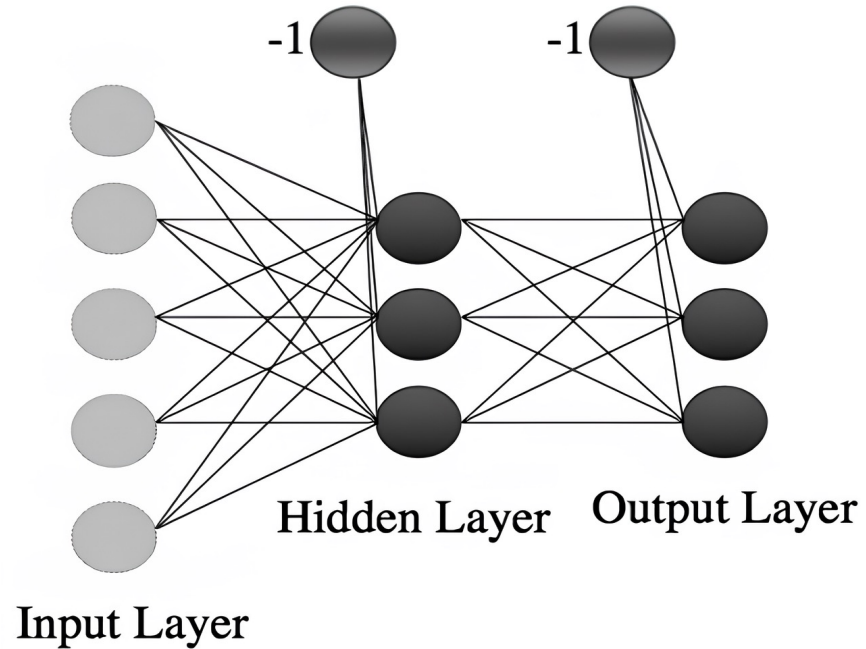
多层神经网络隐层处理后的特征空间

隐藏层神经元实际为特征检测算子
(feature detector), 在多层神经网络的学习过程中, 隐藏层神经元开始逐步“发现”
刻画训练数据的突出特征

A diagram illustrating the transformation from input data to the hidden layer feature space. A green arrow points from the input data table below to the scatter plot above. Another green arrow points from the scatter plot above to the single-layer feature space plot on the left.

0	0	0	1	0
0	1	0	0	1
1	0	0	0	1
1	1	1	0	0

多层感知机的工作机制



计算误差

**向前：分阶段，
逐层计算输入和输出**

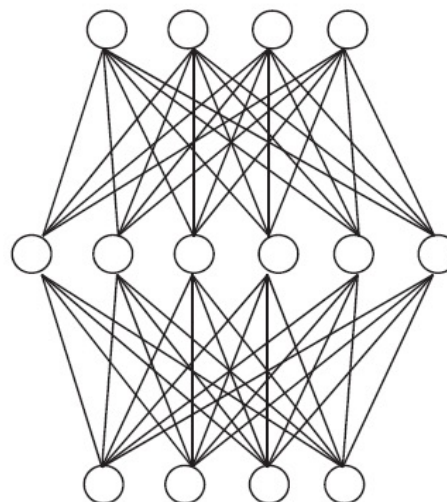
**向后：分阶段，
逐层调整权值**

多层前馈神经网络

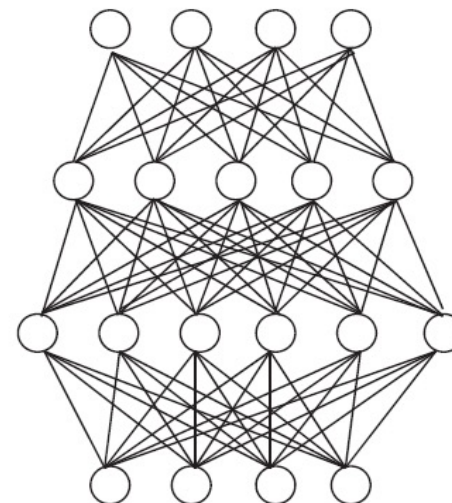
多层网络：包含隐层的网络

前馈网络：神经元之间不存在同层连接也不存在跨层连接

隐层和输出层神经元亦称“功能单元”(functional unit)



(a) 单隐层前馈网络



(b) 双隐层前馈网络

多层前馈网络有强大的表示能力 (**“万有逼近性”**)

仅需一个包含足够多神经元的隐层, 多层前馈神经网络就能以任意精度逼近任意复杂度的连续函数 [Hornik et al., 1989]

如何设置隐层神经元数是未决问题(Open Problem), 实际常用“试错法”

大纲

- 脑和神经元
- 激活函数
- 感知机学习
- 多层感知机
- 反向传播算法

误差逆传播算法 (BackPropagation)

□ 迄今最成功、最常用的神经网络算法，可用于多种任务（不限于分类）

(P. Werbos在博士学位论文中正式完整描述: P. Werbos. Beyond regression: New tools for prediction and analysis in the behavioral science. Ph.D dissertation, Harvard University, 1974)

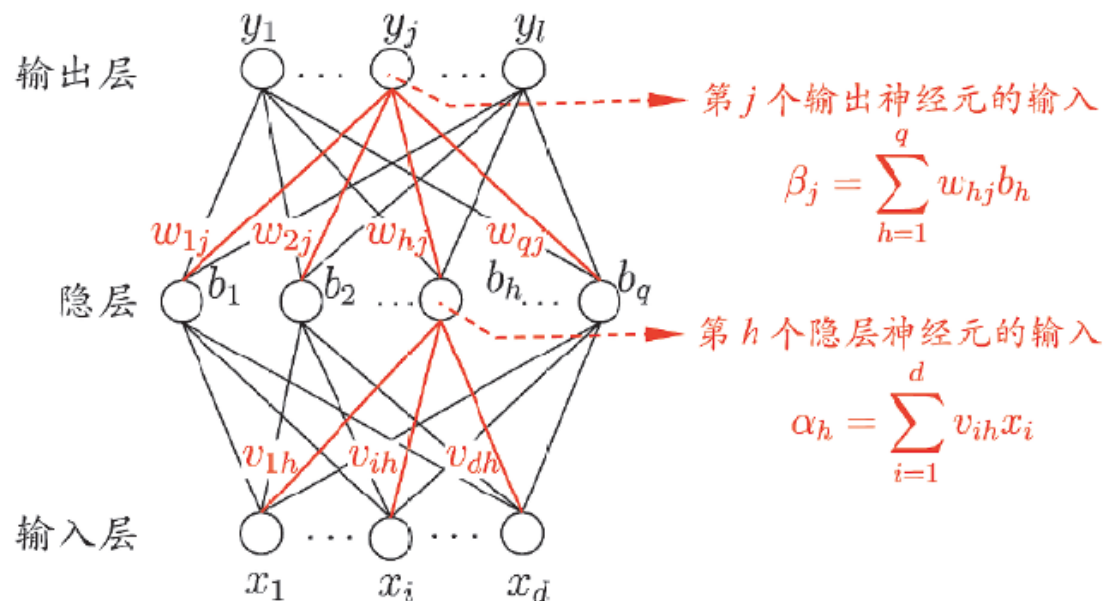
给定训练集 $D = \{(\mathbf{x}_1, \mathbf{y}_1), (\mathbf{x}_2, \mathbf{y}_2), \dots, (\mathbf{x}_m, \mathbf{y}_m)\}, \mathbf{x}_i \in \mathbb{R}^d, \mathbf{y}_i \in \mathbb{R}^l$

输入: d 维特征向量

输出: l 个输出值

隐层: 假定使用 q 个隐层神经元

假定功能单元均使用Sigmoid 函数



BP算法推导

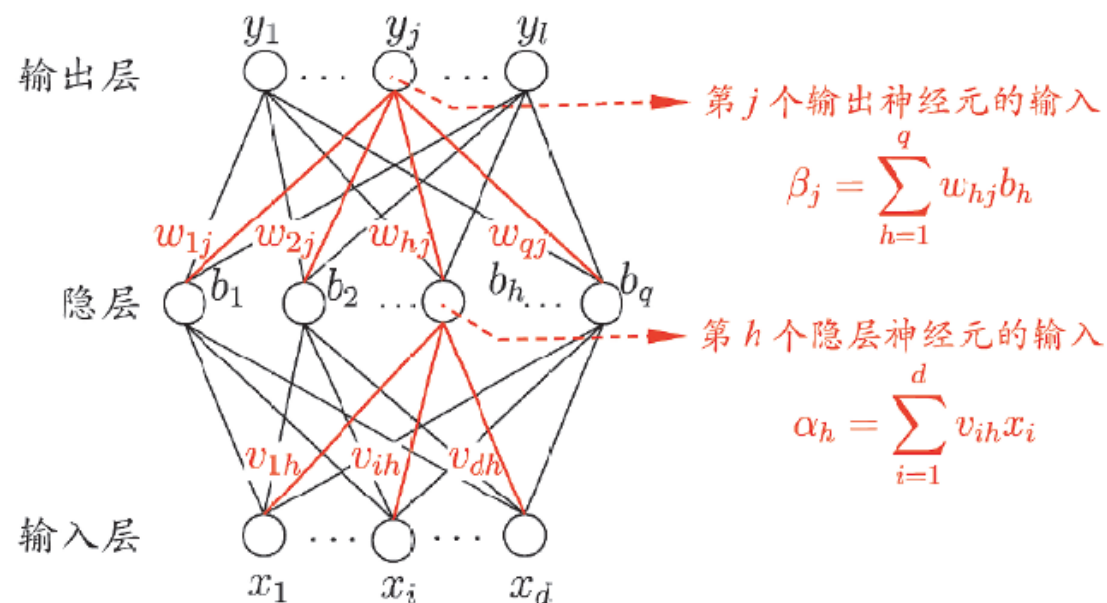
对于训练例 $(\mathbf{x}_k, \mathbf{y}_k)$, 假定网络的实际输出为 $\hat{\mathbf{y}}_k = (\hat{y}_1^k, \hat{y}_2^k, \dots, \hat{y}_l^k)$

$$\hat{y}_j^k = f(\beta_j - \theta_j)$$

则网络在 $(\mathbf{x}_k, \mathbf{y}_k)$ 上的均方误差为:

$$E_k = \frac{1}{2} \sum_{j=1}^l (\hat{y}_j^k - y_j^k)^2$$

需通过学习确定的参数数目: $(d + l + 1)q + l$



BP 是一个迭代学习算法, 在迭代的每一轮中采用广义感知机学习规则

$$v \leftarrow v + \boxed{\Delta v}.$$

BP算法推导

BP 算法基于**梯度下降**策略，以目标的负梯度方向对参数进行调整

以 w_{hj} 为例

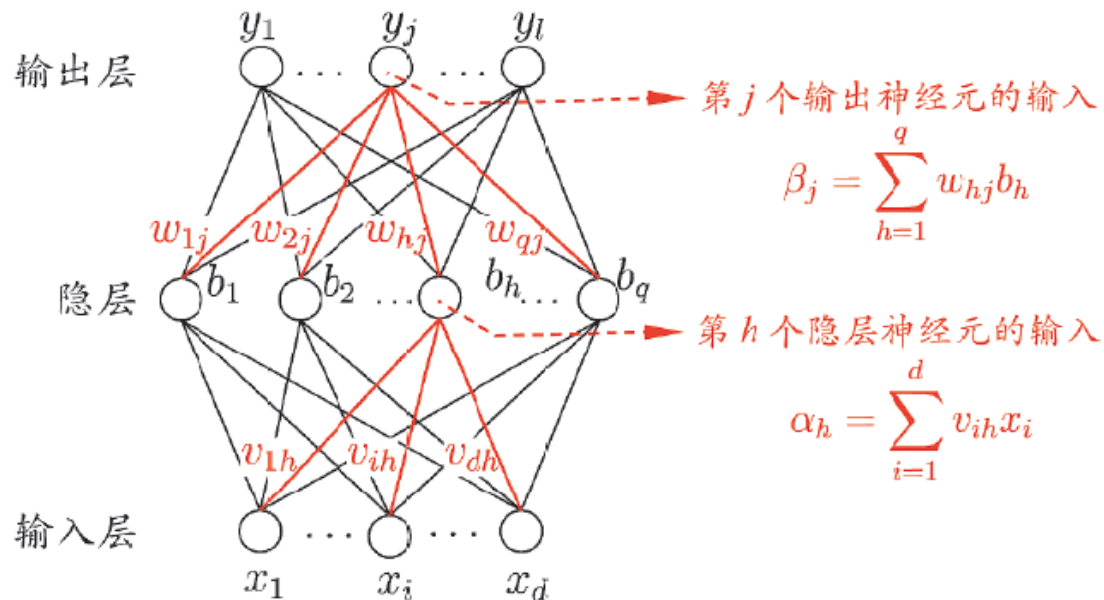
对误差 E_k ，给定学习率 η ，有：

$$\Delta w_{hj} = -\eta \frac{\partial E_k}{\partial w_{hj}}$$

注意到 w_{hj} 先影响到 β_j ，

再影响到 $\hat{y}_j^k$ ，然后才影响到 E_k 有：

$$\frac{\partial E_k}{\partial w_{hj}} = \frac{\partial E_k}{\partial \hat{y}_j^k} \cdot \frac{\partial \hat{y}_j^k}{\partial \beta_j} \cdot \frac{\partial \beta_j}{\partial w_{hj}}$$



← “链式法则”

BP算法推导

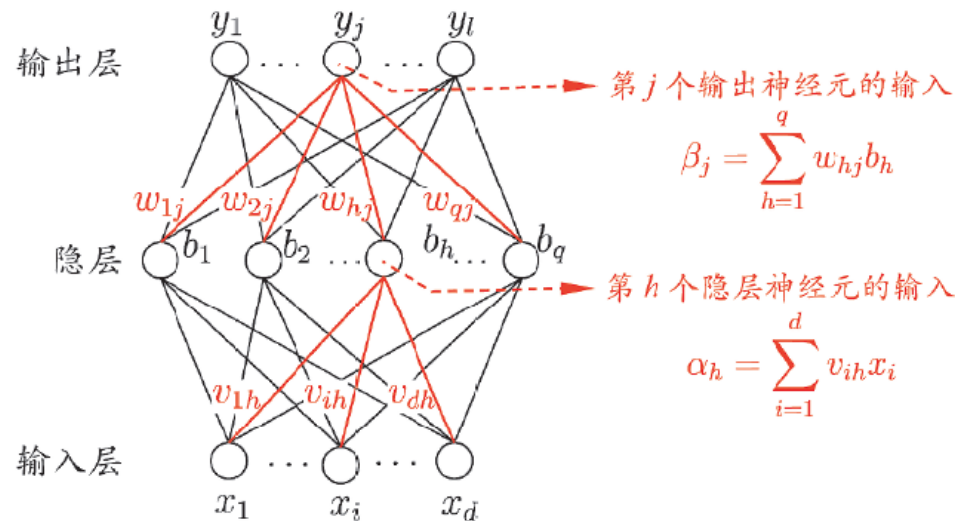
$$\frac{\partial E_k}{\partial w_{hj}} = \underbrace{\frac{\partial E_k}{\partial \hat{y}_j^k}}_{(\hat{y}_j^k - y_j^k)} \cdot \underbrace{\frac{\partial \hat{y}_j^k}{\partial \beta_j}}_{f'(\beta_j - \theta_j)} \cdot \underbrace{\frac{\partial \beta_j}{\partial w_{hj}}}_{= b_h}$$

注意到
 $\hat{y}_j^k = f(\beta_j - \theta_j)$

$$f'(\beta_j - \theta_j)$$

$$\hat{y}_j^k (1 - \hat{y}_j^k)$$

$$\begin{aligned} \text{令 } g_j &= -\frac{\partial E_k}{\partial \hat{y}_j^k} \cdot \frac{\partial \hat{y}_j^k}{\partial \beta_j} \\ &= \hat{y}_j^k (1 - \hat{y}_j^k) (y_j^k - \hat{y}_j^k) \end{aligned}$$



对 $\text{sigmoid}(x) = \frac{1}{1 + e^{-x}}$, 有

$$f'(x) = f(x)(1 - f(x))$$

于是, $\Delta w_{hj} = -\eta \frac{\partial E_k}{\partial w_{hj}} = \eta g_j b_h$

BP算法推导

类似地，有：

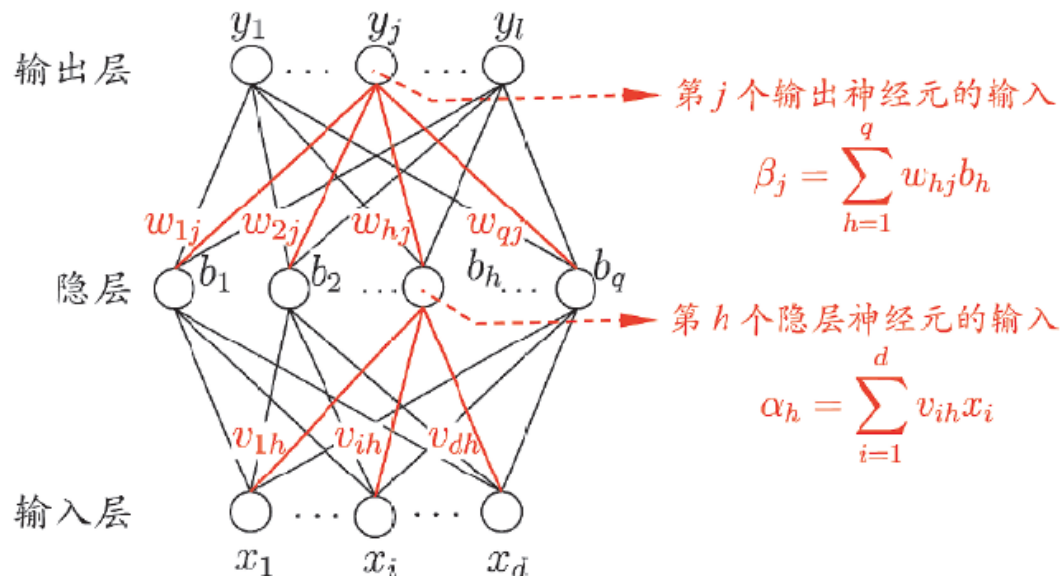
$$\Delta\theta_j = -\eta g_j$$

$$\Delta v_{ih} = \eta e_h x_i$$

$$\Delta\gamma_h = -\eta e_h$$

其中：

$$\begin{aligned} e_h &= -\frac{\partial E_k}{\partial b_h} \cdot \frac{\partial b_h}{\partial \alpha_h} \\ &= -\sum_{j=1}^l \frac{\partial E_k}{\partial \beta_j} \cdot \frac{\partial \beta_j}{\partial b_h} f'(\alpha_h - \gamma_h) = \sum_{j=1}^l w_{hj} g_j f'(\alpha_h - \gamma_h) \\ &= b_h(1 - b_h) \sum_{j=1}^l w_{hj} g_j \end{aligned}$$

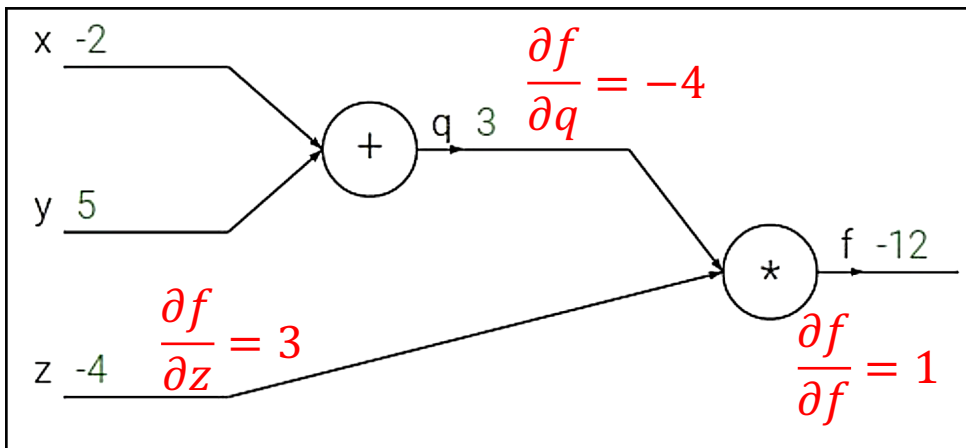


学习率 $\eta \in (0, 1)$ 不能太大、不能太小

BP算法示例

$$f(x, y, z) = (x + y)z$$

$$\text{令 } x = -2, y = 5, z = -4$$



$$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial y} = -4$$

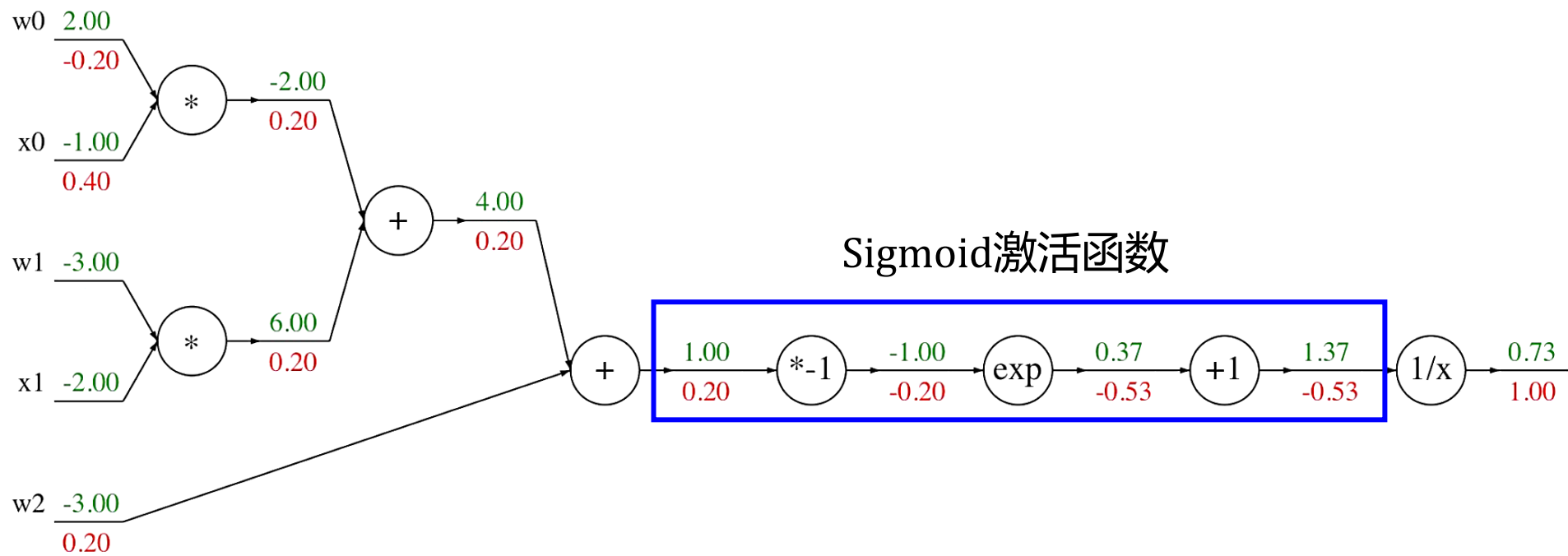
$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial x} = -4$$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

BP算法示例

$$f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}}$$



$f(x) = e^x$	$\rightarrow$	$\frac{df}{dx} = e^x$		$f(x) = \frac{1}{x}$	$\rightarrow$	$\frac{df}{dx} = -1/x^2$
$f_a(x) = ax$	$\rightarrow$	$\frac{df}{dx} = a$		$f_c(x) = c + x$	$\rightarrow$	$\frac{df}{dx} = 1$

标准 BP 算法 vs. 累积 BP 算法

标准 BP 算法

- 每次针对单个训练样例更新权值与阈值
- 参数更新频繁, 不同样例可能抵消, 需要多次迭代

累积 BP 算法

- 其优化目标是 minimized 整个训练集上的累积误差
- 读取整个训练集一遍才对参数进行更新, 参数更新频率较低

在很多任务中, 累积误差下降到一定程度后, 进一步下降会非常缓慢, 这时标准BP算法往往会获得较好的解, 尤其当训练集非常大时效果更明显.

缓解过拟合

主要策略：

□ 早停(early stopping)

- 若训练误差连续 a 轮的变化小于 b , 则停止训练
- 使用验证集：若训练误差降低、验证误差升高, 则停止训练

□ 正则化 (regularization)

- 在误差目标函数中增加一项描述网络复杂度

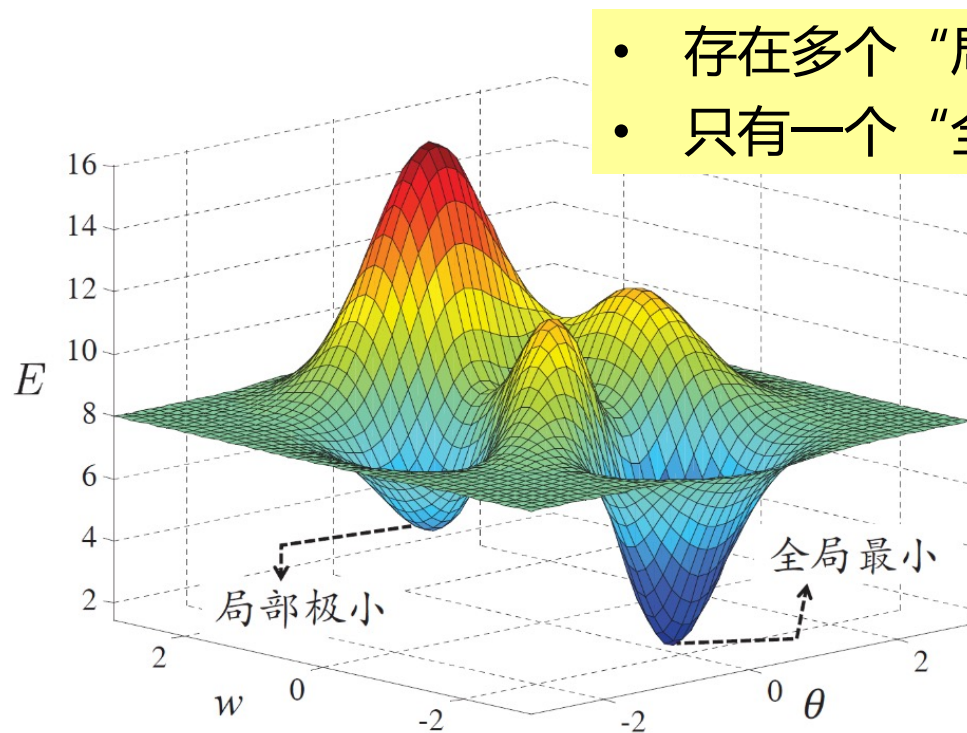
例如
$$E = \lambda \frac{1}{m} \sum_{k=1}^m E_k + (1 - \lambda) \sum_i w_i^2$$

偏好比较小的连接权和阈值,
使网络输出更“光滑”

全局最小 vs. 局部极小

神经网络的训练过程可看作一个参数寻优过程：

在参数空间中，寻找一组最优参数使得误差最小



“跳出”局部极小的常见策略：

- ✓ 不同的初始参数
- ✓ 模拟退火
- ✓ 随机扰动
- ✓ 演化算法
- ✓

谢谢!

联系方式: liwenbin@nju.edu.cn

更多信息: <https://cs.nju.edu.cn/liwenbin>

模型轻量化

Model Lightweighting

助教：李兵

522023330043@smail.nju.edu.cn

2024年4月9日

目录

- **模型轻量化介绍**
- **剪枝**
- **蒸馏**
- **量化**

模型轻量化

□ 模型轻量化的定义

- 模型轻量化 (Model Lightweighting) 是指通过一系列技术手段, 减少机器学习或深度学习模型的复杂度和计算量, 以便在资源受限的环境中部署和运行。它的目标是在保持模型性能的同时, 降低模型的存储需求、计算需求和推理时间。

□ 模型轻量化提出了三部分优化

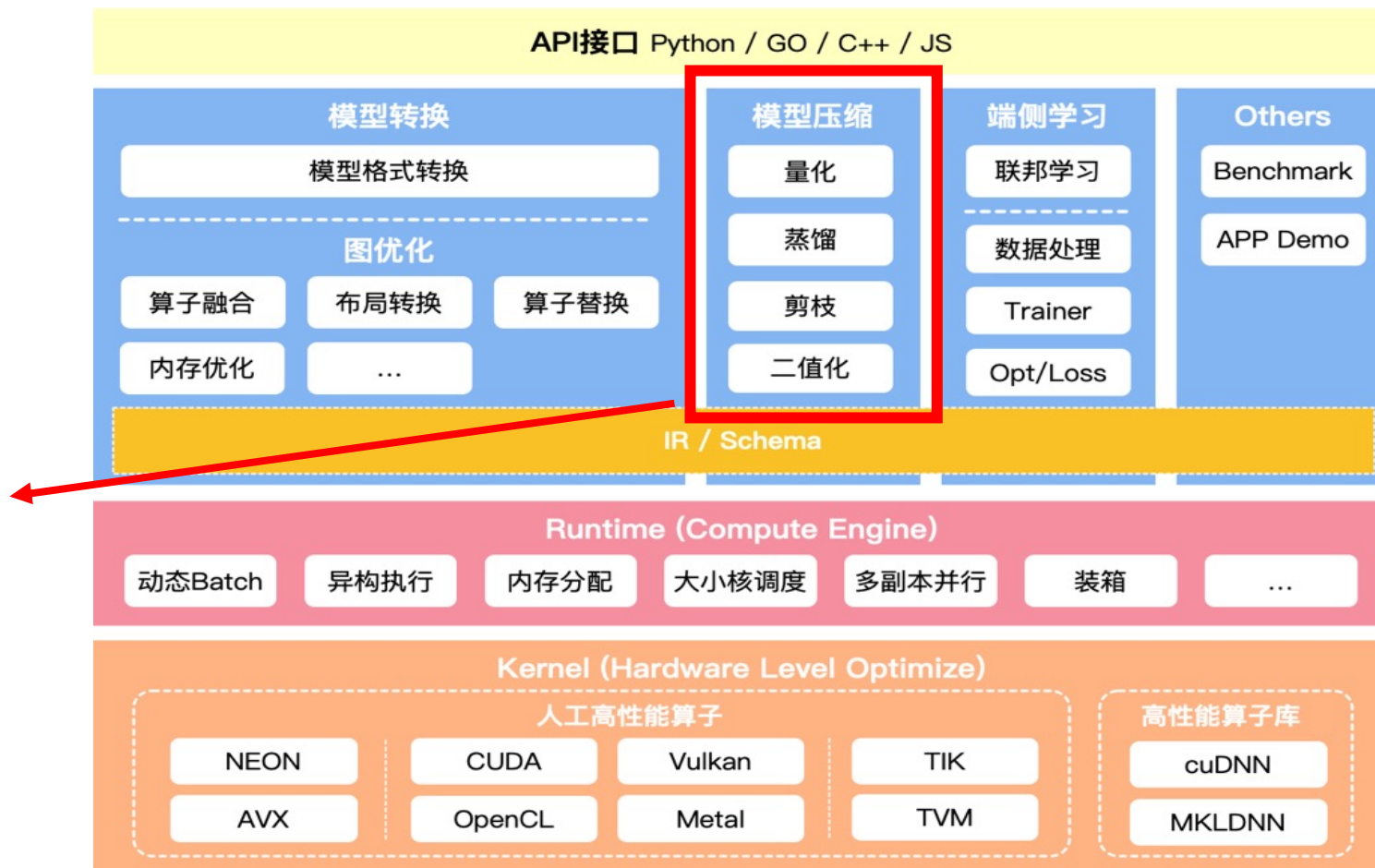
- 减少内存密集访问量 (减少内存访问时间)
- 提高获取模型密集参数的时间 (模型参数越少越好, 模型越小越好)
- 加速模型推理时间 (模型跑得越快越好)

模型轻量化

□ 模型轻量化（模型压缩，模型小型化）

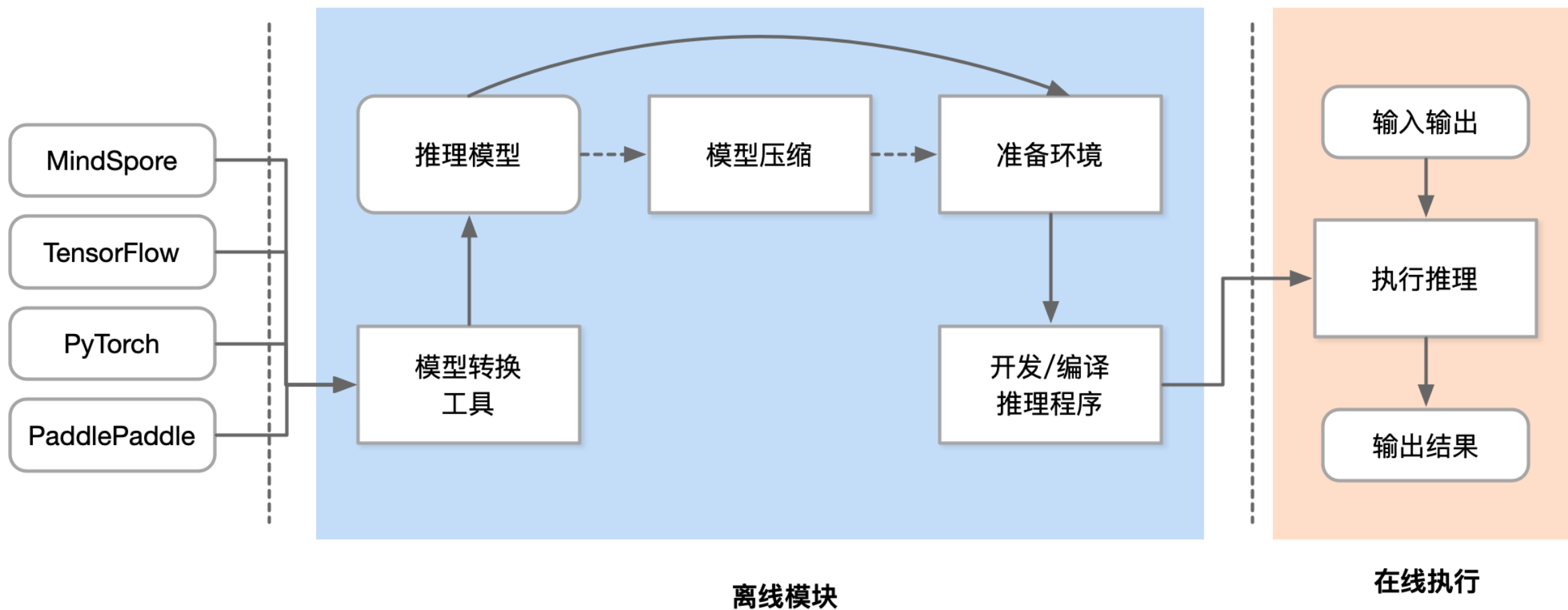
- 1.减少模型大小
- 2.加快推理速度
- 3.保持相同精度

多维混合压缩算法



模型轻量化

□ 推理引擎架构



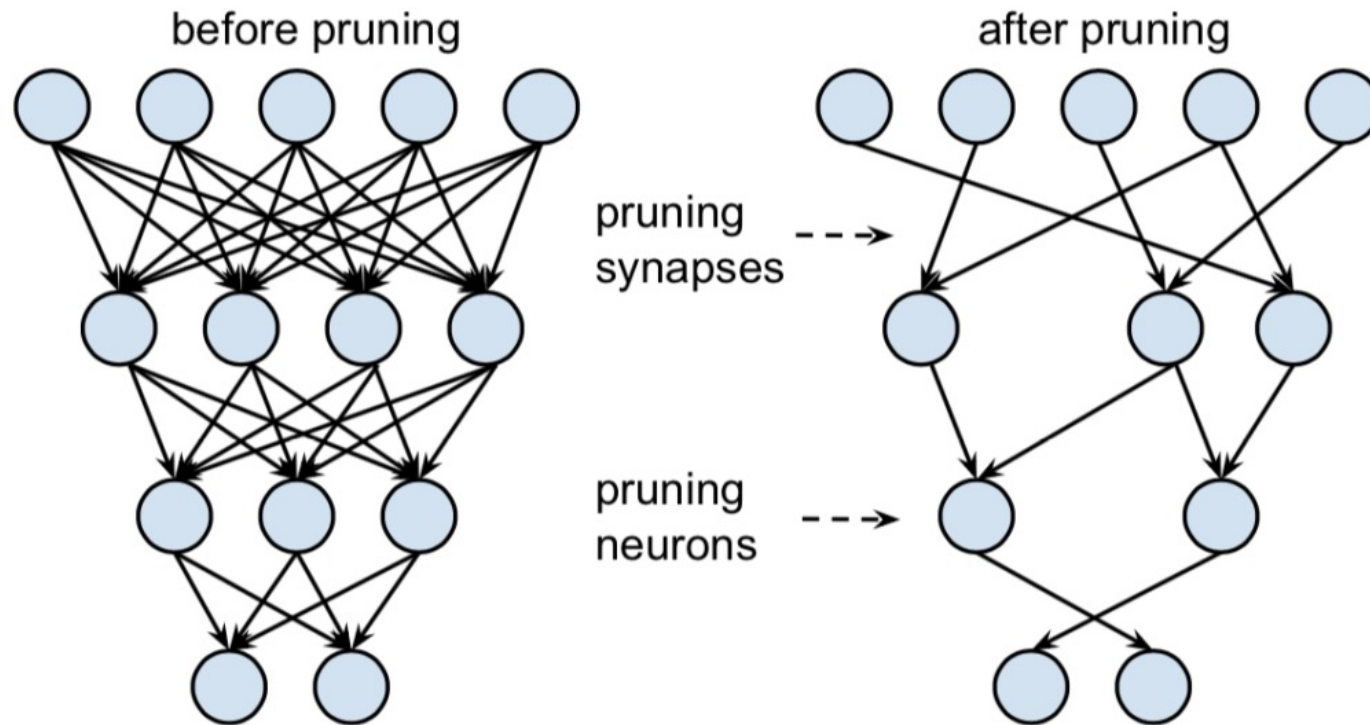
目录

- 模型轻量化介绍
- 剪枝
- 蒸馏
- 量化

剪枝

□ 模型剪枝

- 模型剪枝研究模型权重中的冗余，并尝试删除/修剪冗余和非关键的权重



剪枝

□ 模型剪枝

- 模型剪枝研究模型权重中的冗余，并尝试删除/修剪冗余和非关键的权重
- To prune or not to prune

1. 内存占用相同，大稀疏模型比小密集模型实现了更高的精度。
2. 剪枝之后稀疏模型优于同体积非稀疏模型。
3. 资源有限，剪枝是比较有效的模型压缩策略。
4. 还可以优化硬件稀疏矩阵储存方向。

Table 4: NMT sparse vs dense results

# units	Sparsity	NNZ params	EN-DE BLEU score	DE-EN BLEU score
256	0%	34M	23.52	26.52
512	0%	81M	26.05	28.88
768	0%	140M	26.63	29.41
1024	0%	211M	26.77	29.47
	80%	44M	26.86	29.50
	85%	33M	26.52	29.24
	90%	23M	26.19	28.81

Table 1: Model size and accuracy tradeoff for sparse-InceptionV3

Sparsity	NNZ params	Top-1 acc.	Top-5 acc.
0%	27.1M	78.1%	94.3%
50%	13.6M	78.0%	94.2%
75%	6.8M	76.1%	93.2%
87.5%	3.3M	74.6%	92.5%

剪枝

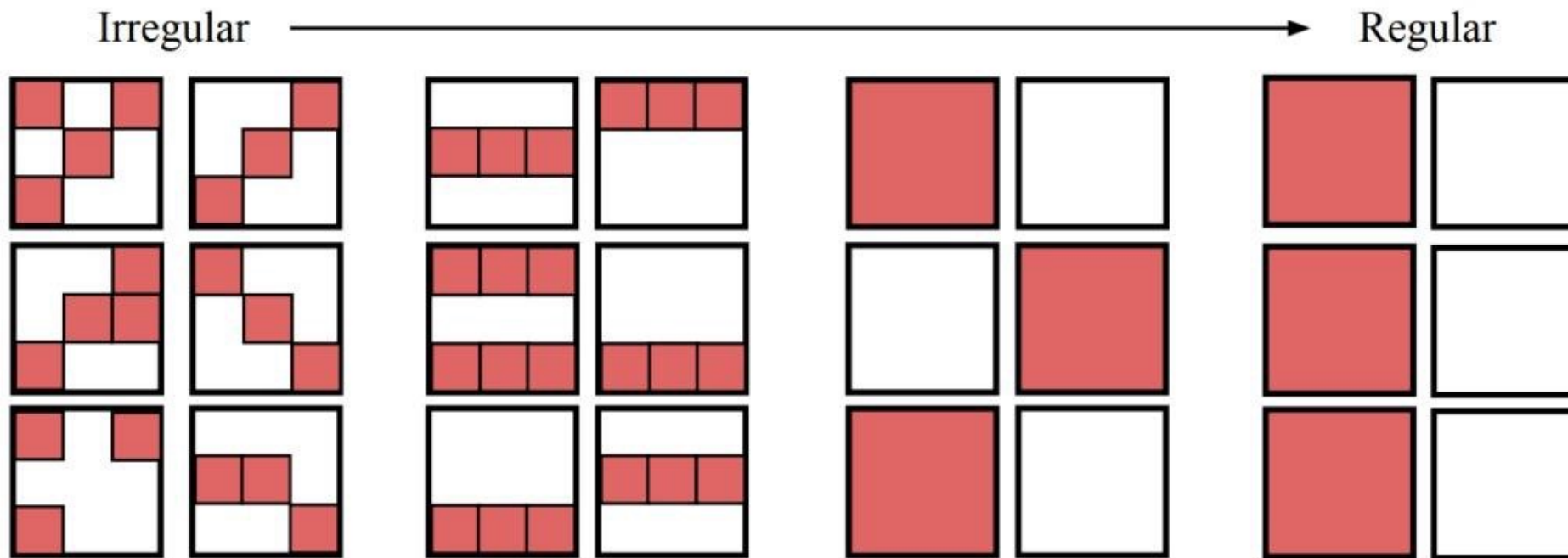
□ 剪枝算法

Name	Brief Introduction of Algorithm
Level Pruner	Pruning the specified ratio on each weight element based on absolute value of weight element
L1 Norm Pruner	Pruning output channels with the smallest L1 norm of weights (Pruning Filters for Efficient Convnets) Reference Paper
L2 Norm Pruner	Pruning output channels with the smallest L2 norm of weights
FPGM Pruner	Filter Pruning via Geometric Median for Deep Convolutional Neural Networks Acceleration Reference Paper
Activation Mean Rank Pruner	Pruning output channels based on the metric that calculates the smallest mean value of output activations
Taylor FO Weight Pruner	Pruning filters based on the first order taylor expansion on weights(Importance Estimation for Neural Network Pruning) Reference Paper
ADMM Pruner	Pruning based on ADMM optimization technique Reference Paper
Linear Pruner	Sparsity ratio increases linearly during each pruning rounds, in each round, using a basic pruner to prune the model.
AGP Pruner	Automated gradual pruning (To prune, or not to prune: exploring the efficacy of pruning for model compression) Reference Paper
Simulated Annealing Pruner	Automatic pruning with a guided heuristic search method, Simulated Annealing algorithm Reference Paper
Auto Compress Pruner	Automatic pruning by iteratively call SimulatedAnnealing Pruner and ADMM Pruner Reference Paper
AMC Pruner	AMC: AutoML for Model Compression and Acceleration on Mobile Devices Reference Paper

剪枝

□ 模型剪枝分类

- 结构化剪枝（对filter/channel/layer进行剪枝）
- 非结构化剪枝（随机对独立的权重或者神经元进行剪枝）



剪枝

□ 模型剪枝分类

- 结构化剪枝（对filter/channel/layer进行剪枝）
 - 优点：大部分算法都保留了原始结构，使用一般硬件也可以实现
 - 剪枝的算法比较复杂
- 非结构化剪枝（随机对独立的权重或者神经元进行剪枝）
 - 优点：剪枝算法简单，模型压缩比高
 - 缺点：精度不可控制，需要使用专有硬件进行加速。

剪枝

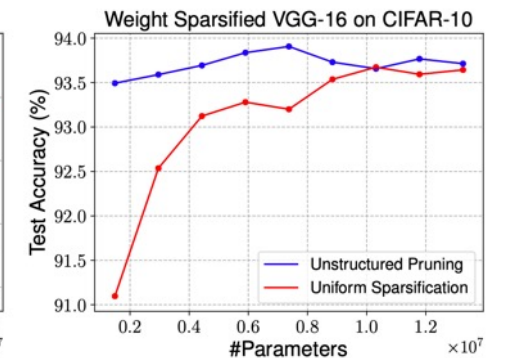
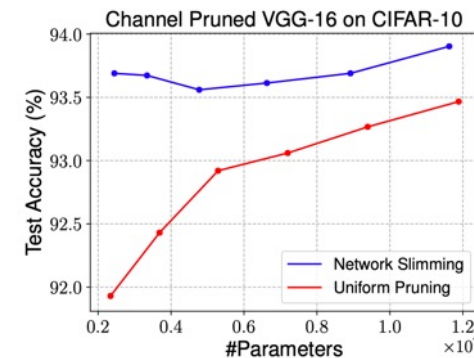
□ 算法举例 (L1-norm based Channel Pruning)

- 算法步骤
 - 1.对每个卷积核, 计算权重绝对值 (L1-norm) 之和
 - 2.根据L1-norm的值进行排序
 - 3.将m个权重绝对值之和最小的卷积核以及对应的feature maps进行剪枝
 - 4.下一个卷积层里面与feature maps相关的卷积核进行剪枝
 - 5.下面层的新权重矩阵被创建, 剩下的权重参数被复制到新的模型

剪枝

□ L1-norm based Channel Pruning

Dataset	Model	Unpruned	Prune Ratio	Fine-tuned	Scratch-E	Scratch-B
CIFAR-10	VGG-19	93.50 (± 0.11)	30%	93.51 (± 0.05)	93.71 (± 0.09)	93.31 (± 0.26)
			80%	93.52 (± 0.10)	93.71 (± 0.08)	93.64 (± 0.09)
			95%	93.34 (± 0.13)	93.21 (± 0.17)	93.63 (± 0.18)
	PreResNet-110	95.04 (± 0.15)	30%	95.06 (± 0.05)	94.84 (± 0.07)	95.11 (± 0.09)
			80%	94.55 (± 0.11)	93.76 (± 0.10)	94.52 (± 0.13)
			95%	92.35 (± 0.20)	91.23 (± 0.11)	91.55 (± 0.34)
	DenseNet-BC-100	95.24 (± 0.17)	30%	95.21 (± 0.17)	95.22 (± 0.18)	95.23 (± 0.14)
			80%	95.04 (± 0.15)	94.42 (± 0.12)	95.12 (± 0.04)
			95%	94.19 (± 0.15)	92.91 (± 0.22)	93.44 (± 0.19)
CIFAR-100	VGG-19	71.70 (± 0.31)	30%	71.96 (± 0.36)	72.81 (± 0.31)	73.30 (± 0.25)
			50%	71.85 (± 0.30)	73.12 (± 0.36)	73.77 (± 0.23)
			95%	70.22 (± 0.38)	70.88 (± 0.35)	72.08 (± 0.15)
	PreResNet-110	76.96 (± 0.34)	30%	76.88 (± 0.31)	76.36 (± 0.26)	76.96 (± 0.31)
			50%	76.60 (± 0.36)	75.45 (± 0.23)	76.42 (± 0.39)
			95%	68.55 (± 0.51)	68.13 (± 0.64)	68.99 (± 0.32)
	DenseNet-BC-100	77.59 (± 0.19)	30%	77.23 (± 0.05)	77.58 (± 0.25)	77.97 (± 0.31)
			50%	77.41 (± 0.14)	77.65 (± 0.09)	77.80 (± 0.23)
			95%	73.67 (± 0.03)	71.47 (± 0.46)	72.57 (± 0.37)
ImageNet	VGG-16	73.37	30%	73.68	72.75	74.02
			60%	73.63	71.50	73.42
	ResNet-50	76.15	30%	76.06	74.77	75.70
			60%	76.09	73.69	74.91



剪枝率达到95%，精度下降不到2%。

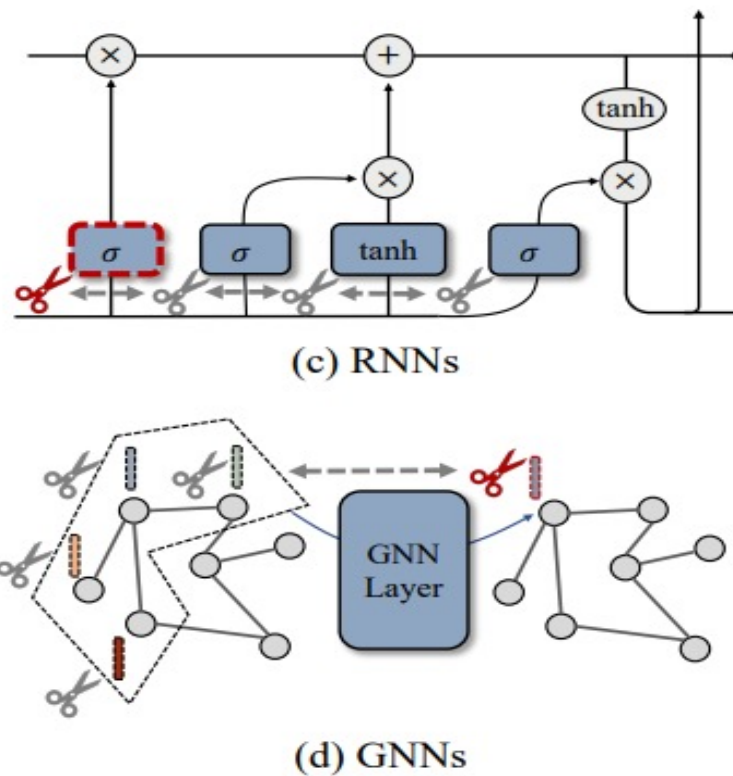
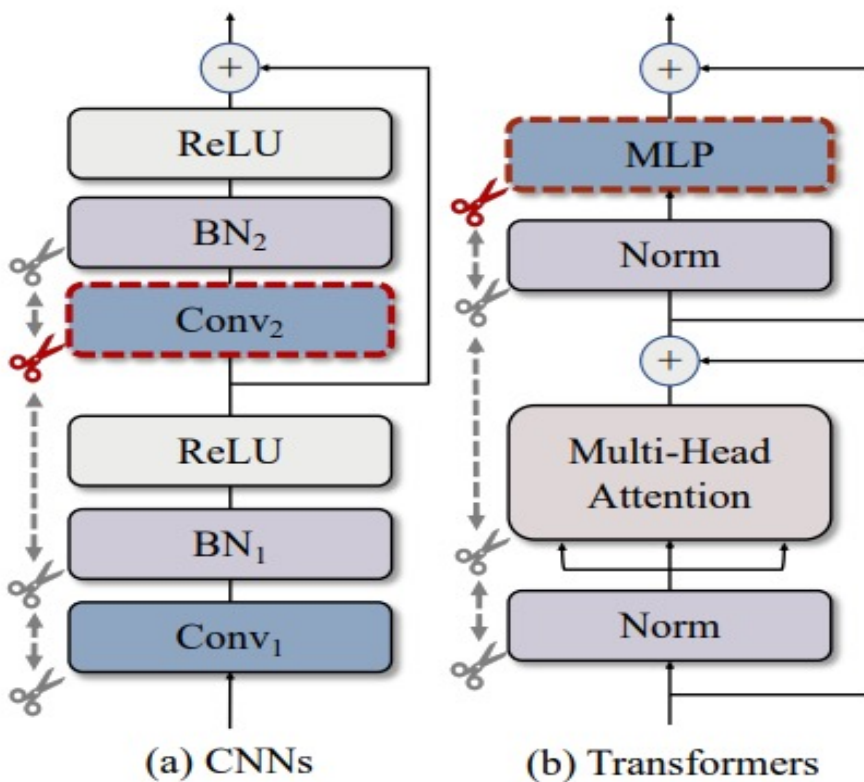
Table 6: Results (accuracy) for unstructured pruning (Han et al., 2015). “Prune Ratio” denotes the percentage of parameters pruned in the set of all convolutional weights.

剪枝

□ 相关技术（结构化剪枝）

- DepGraph: Towards Any Structural Pruning CVPR2023

([\[2301.12900\] DepGraph: Towards Any Structural Pruning \(arxiv.org\)](#))

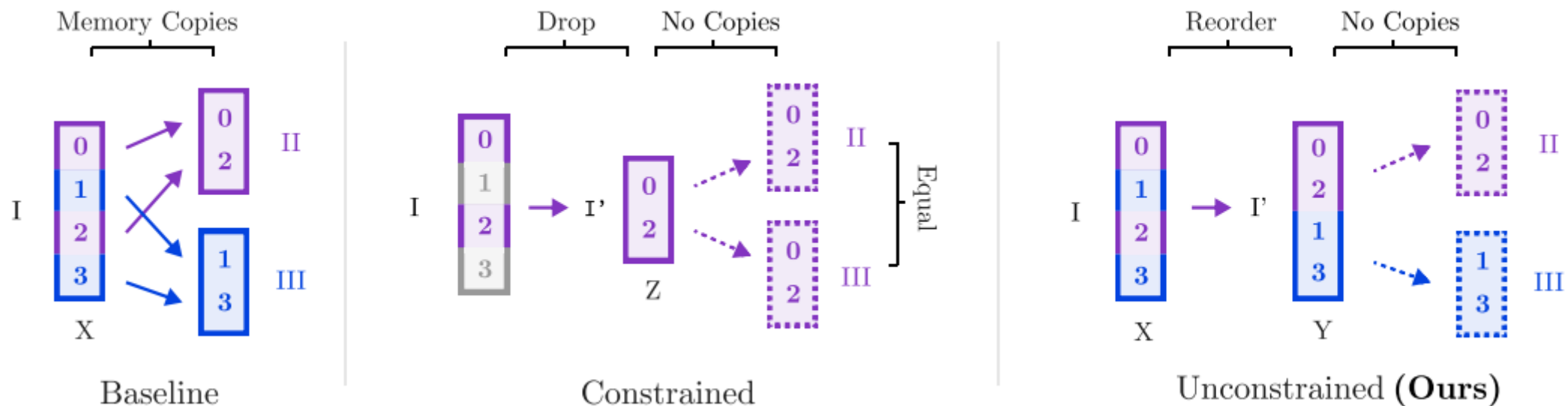


剪枝

□ 相关技术（结构化剪枝）

- **UPSCALE: Unconstrained Channel Pruning** ICML2023

([2307.08771.pdf \(arxiv.org\)](https://arxiv.org/pdf/2307.08771.pdf))

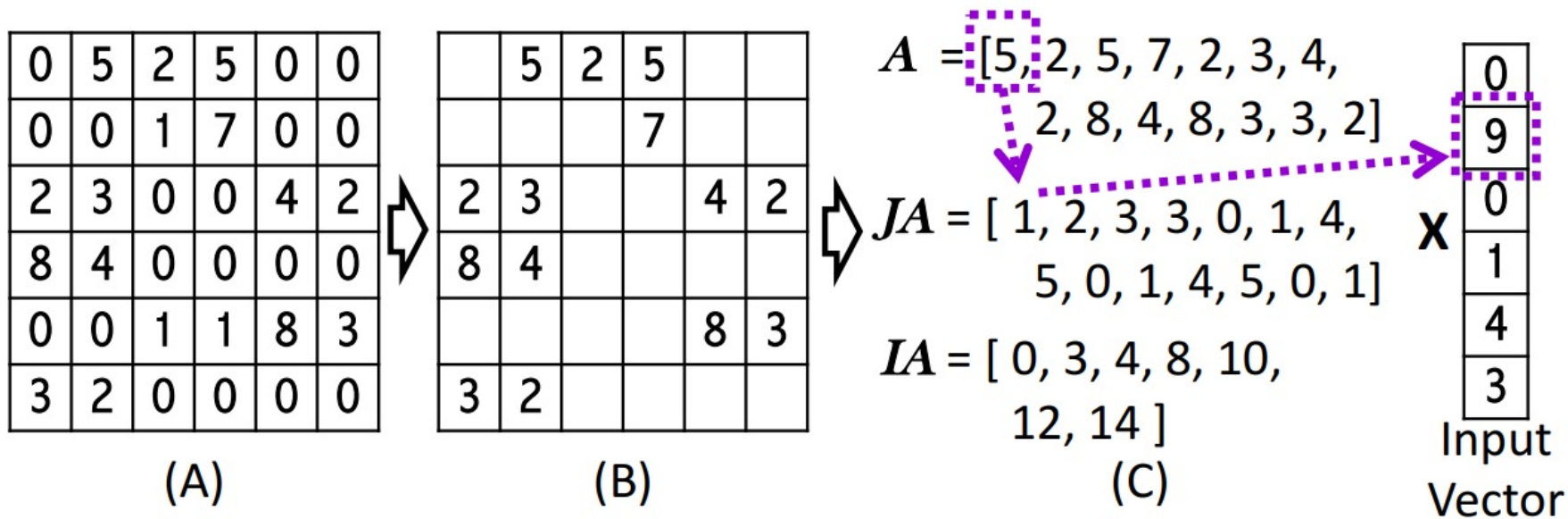


剪枝

□ 相关技术（非结构化剪枝）

- **Scalpel: Customizing DNN pruning to the underlying hardware parallelism** Publisher

([Scalpel: Customizing DNN pruning to the underlying hardware parallelism | IEEE Conference Publication | IEEE Xplore](#))



剪枝

□ 相关技术

- **Structural Pruning for Diffusion Models (Neurips 2024)**
([\[2305.10924\] Structural Pruning for Diffusion Models \(arxiv.org\)](#))
- **Fluctuation-based Adaptive Structured Pruning for Large Language Models**
([\[2305.10924\] Structural Pruning for Diffusion Models \(arxiv.org\)](#))

目录

- 模型轻量化介绍
- 剪枝
- 蒸馏
- 量化

蒸馏

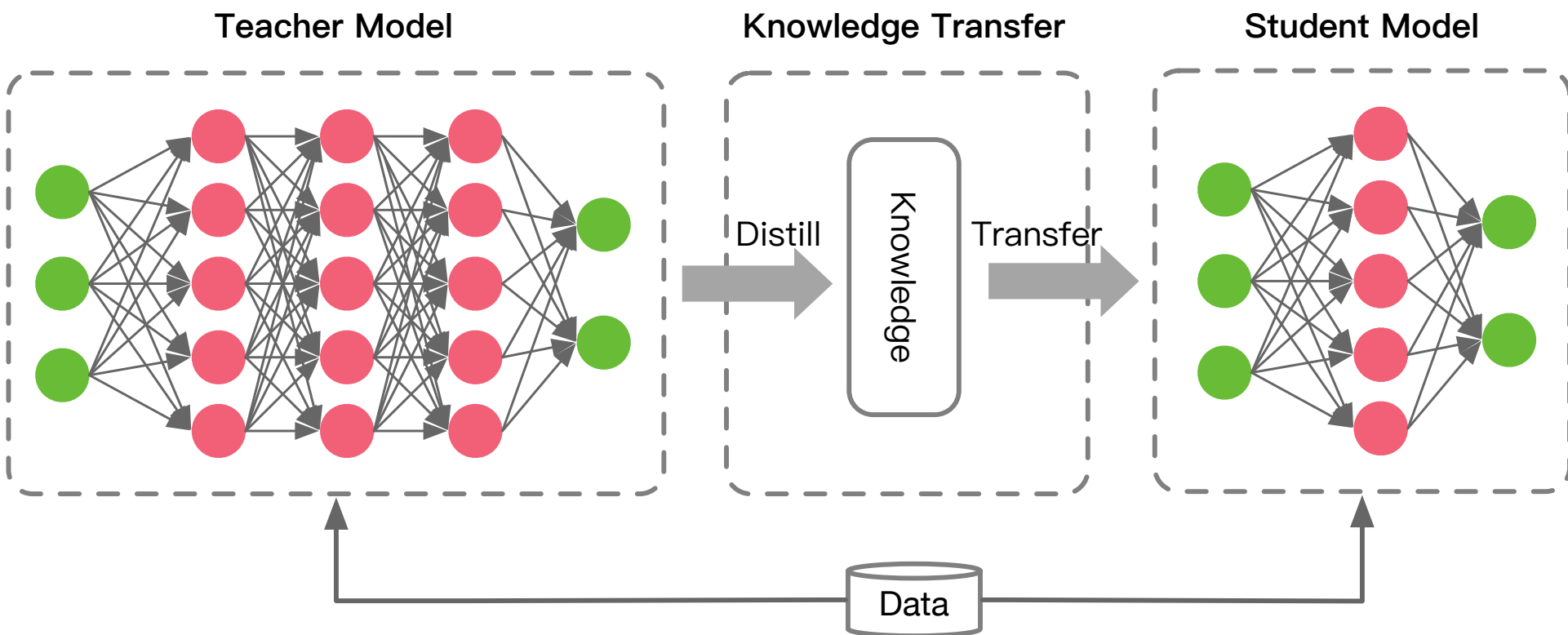
□ 背景

- Knowledge Distillation (KD) 最初是 Hinton 在 “Distilling the Knowledge in a Neural Network”提出，与 Label smoothing 动机类似，但是 KD 生成 soft label 方式通过教师网络得到。
- KD 可以视为将教师网络学到的知识压缩到学生网络中，另外一些工作 “Circumventing outlier of auto augment with knowledge distillation”则将 KD 视为数据增强方法的一种。

蒸馏

□ 算法

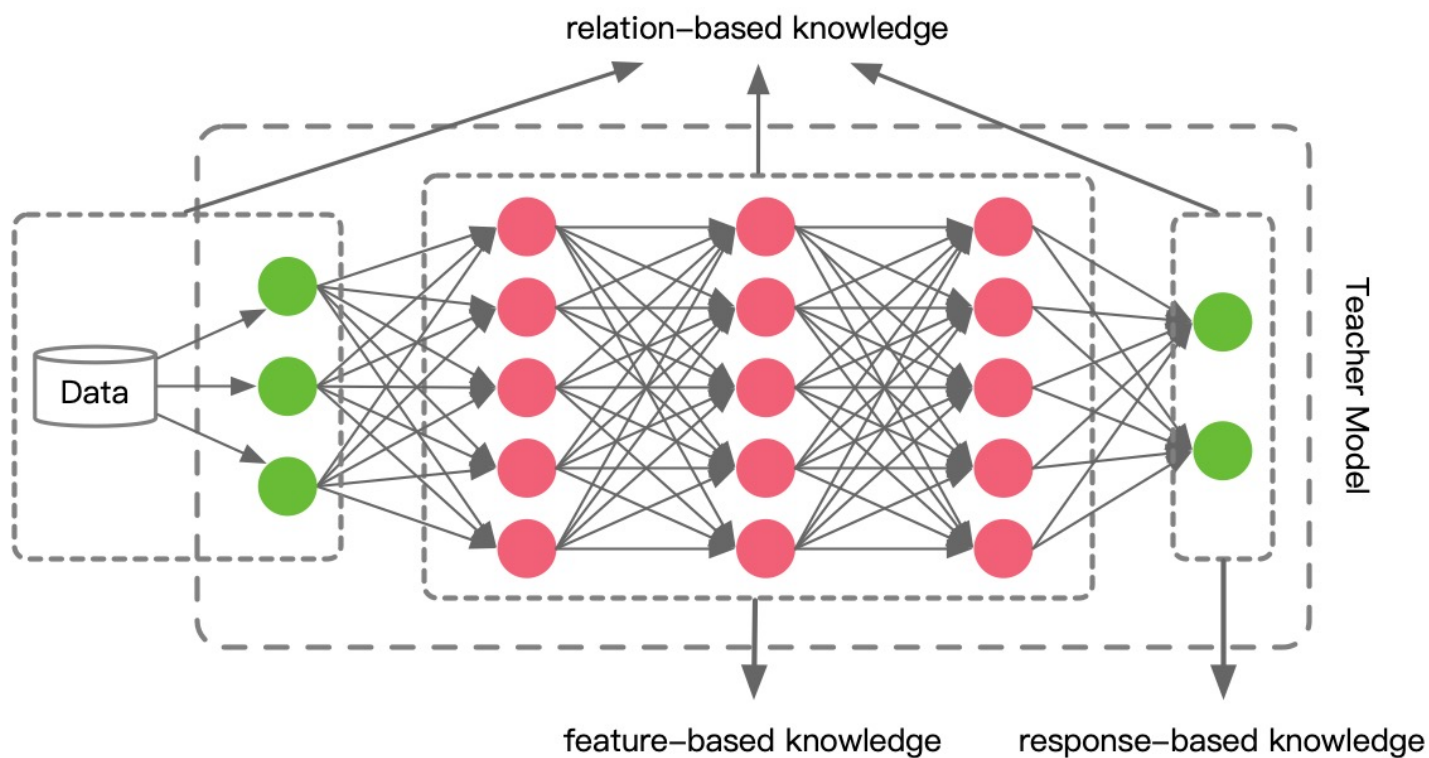
- 由：1) 知识 Knowledge、2) 蒸馏算法 Distillate、3) 师生架构三个关键部分组成：



蒸馏

□ 知识 (Knowledge)

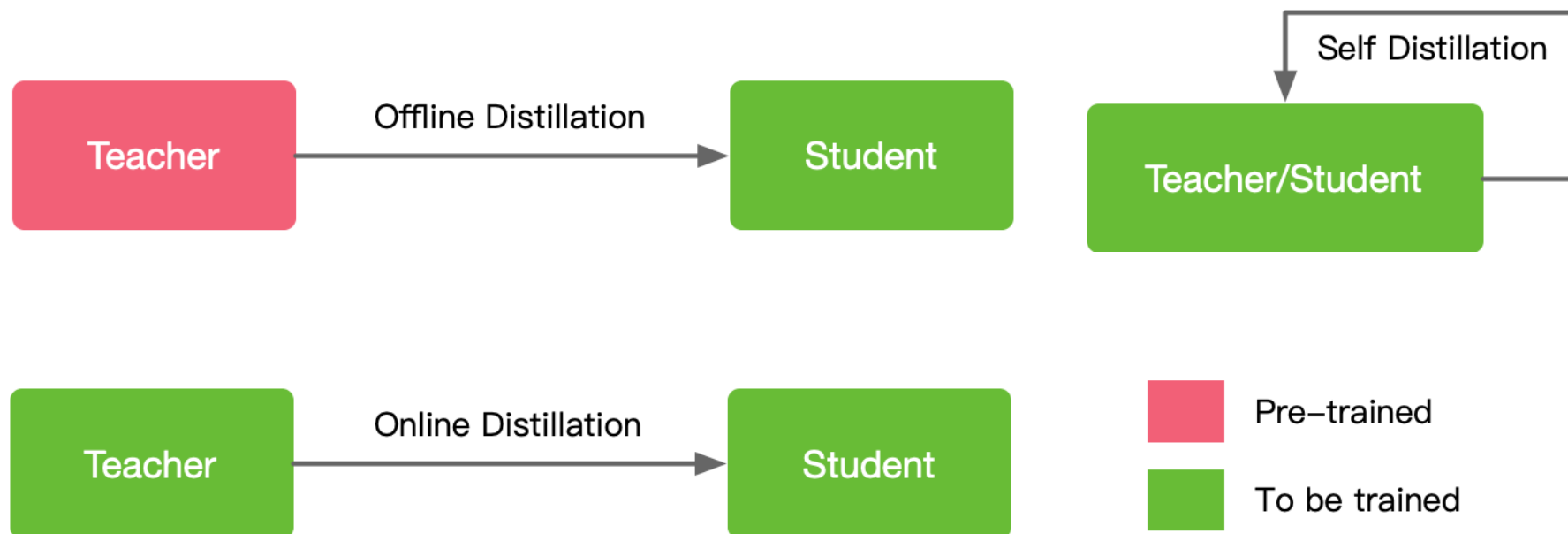
- 1 Response relation feature Architecture
- 2 feature-based knowledge
- 3 relation-based knowledge
- 4 Architecture-base knowledge



蒸馏

□ 蒸馏

- Offline distillation (知识渊博的老师传授学生知识)
- Online distillation (教师和学生共同学习知识)
- Self-distillation (学生自己学习知识)



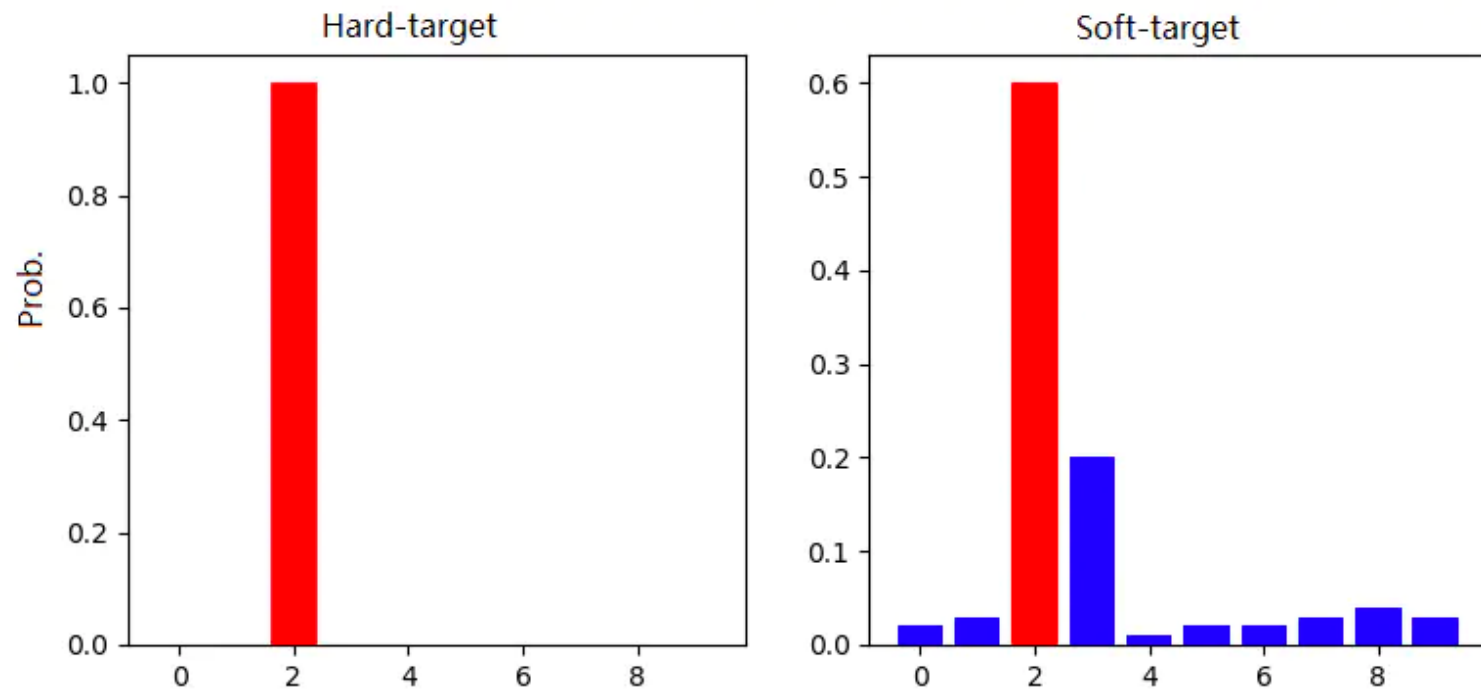
蒸馏

□ 蒸馏

- Distilling the Knowledge in a Neural Network (<https://arxiv.org/pdf/1503.02531.pdf>)

$$q_i = \frac{\exp(z_i)}{\sum_j \exp(z_j)}$$

$$q_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$$



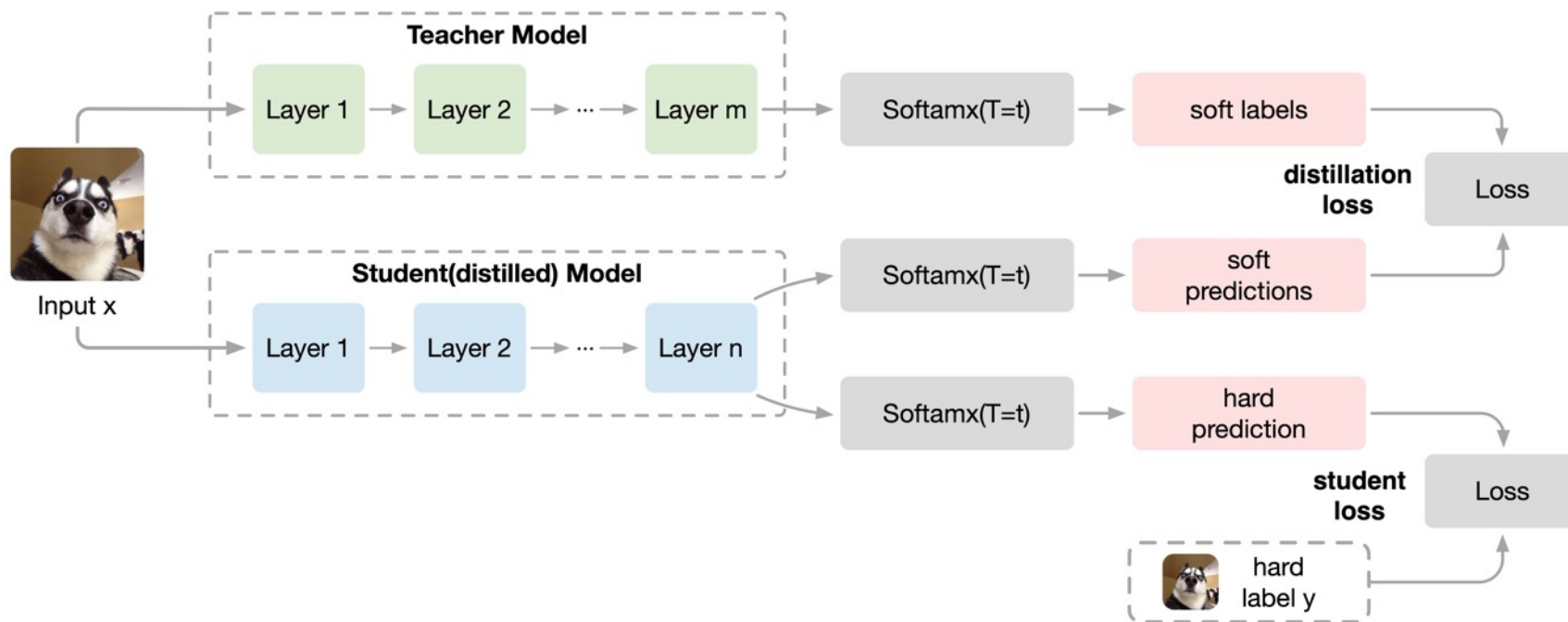
蒸馏

□ 蒸馏

- **Distilling the Knowledge in a Neural Network** (<https://arxiv.org/pdf/1503.02531.pdf>)

传统training过程 Hard Targets: 对 ground truth 求极大似然 Softmax 值。

KD的training过程 Soft Targets: 用 Teacher 模型的 class probabilities作为soft targets。



蒸馏

□ 蒸馏

- **Distilling the Knowledge in a Neural Network(2016)**
(<https://arxiv.org/pdf/1503.02531.pdf>)
- **Training data-efficient image transformers & distillation through attention(ICML 2021)**
(<https://proceedings.mlr.press/v139/touvron21a/touvron21a.pdf>)
- **One-for-All: Bridge the Gap Between Heterogeneous Architectures in Knowledge Distillation**
(NeurIPS2023)
(<https://link.zhihu.com/?target=https%3A//arxiv.org/pdf/2310.19444>)
- **Weight Averaging Improves Knowledge Distillation under Domain Shift(ICCV2023)**
(<https://link.zhihu.com/?target=https%3A//arxiv.org/pdf/2309.11446.pdf>)

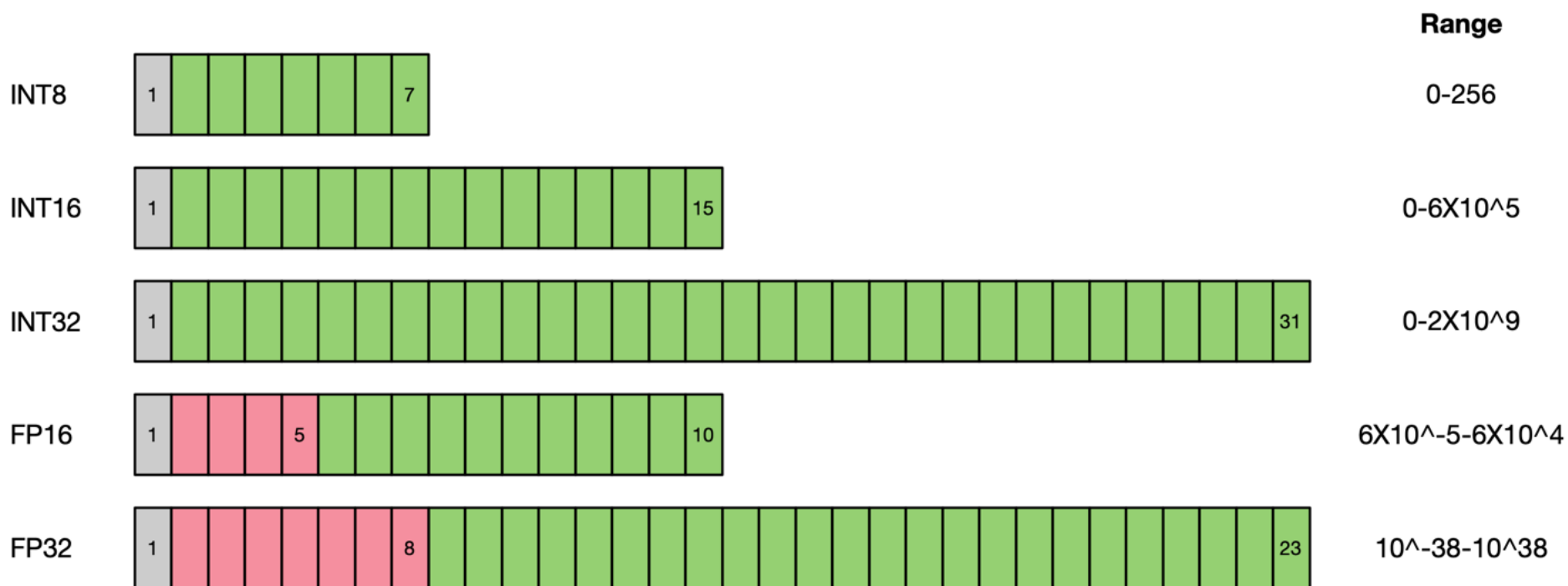
目录

- 模型轻量化介绍
- 剪枝
- 蒸馏
- 量化

量化

□ 模型量化

- 模型量化是一种将浮点计算转成低比特定点计算的技术，可以有效的降低模型计算强度、参数大小和内存消耗，但往往带来巨大的精度损失。尤其是在极低比特(<4bit)、二值网络(1bit)、甚至将梯度进行量化时，带来的精度挑战更大。



量化

□ 模型量化的优点

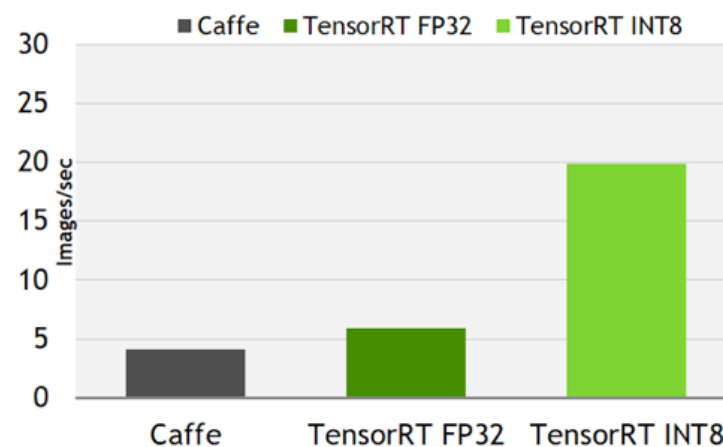
- 保持精度：量化会损失精度，这相当于给网络引入了噪声，但是神经网络一般对噪声是不太敏感的，只要控制好量化的程度，对高级任务精度影响可以做到很小。
- 加速计算：传统的卷积操作都是使用FP32浮点，低比特的位数减少少计算性能也更高，INT8相对比 FP32 的加速比可达到3倍甚至更高。
- 节省内存：与 FP32 类型相比，FP16、INT8、INT4 低精度类型所占用空间更小，对应存储空间和传输时间都可以大幅下降。
- 节能和减少芯片面积：每个数使用了更少的位数，做运算时需要搬运的数据量少了，减少了访存开销（节能），同时所需的乘法器数目也减少（减少芯片面积）。

量化

□ 模型量化主要的技术挑战

- 精度挑战：量化方法，低比特，任务，大小
- 硬件支持程度：支持低bit指令不同，提供的低比特指令计算方式不同，体系的优化方法不同
- 软件算法是否能加速： 1.量化与反量化。2.降低内存与模型参数之间的差异。3.参数量小，压缩比高=? 执行内存占用小。

	CAFFE	TENSORRT FP32	TENSORRT INT8
Runtime (ms)	242	170	50
Images/sec	4	6	20
Class IoU	48.4	48.4	48.1
Category IoU	76.9	76.9	76.8



Batch Size = 1, Input/Output Resolution = 512 x 1024

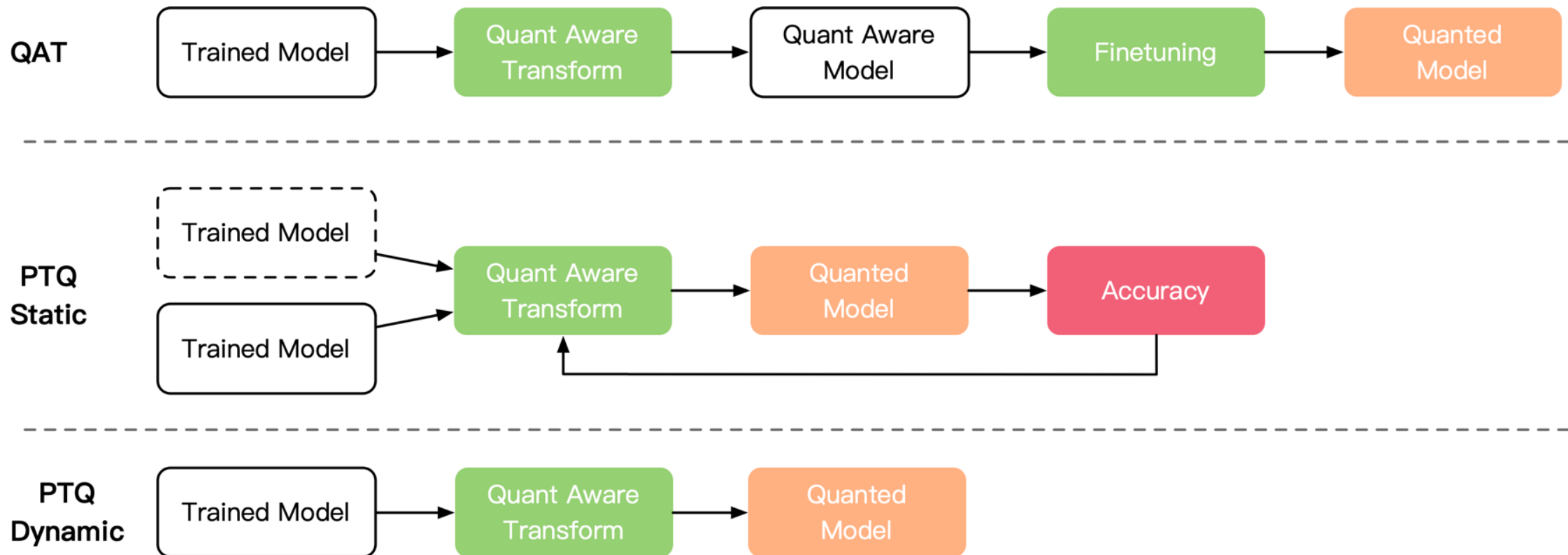
量化

□ 量化方法

- **量化训练(Quant Aware Training, QAT)**: 量化训练让模型感知量化运算对模型精度带来的影响, 通过 finetune 训练降低量化误差。
- **动态离线量化 (Post Training Quantization Dynamic, PTQ Dynamic)**: 动态离线量化仅将模型中特定算子的权重从FP32类型映射成 INT8/16 类型。
- **静态离线量化 (Post Training Quantization Static, PTQ Static)**: 静态离线量化使用少量无标签校准数据, 采用 KL 散度等方法计算量化比例因子。

量化

□ 量化方法



量化

□ 量化方法比较

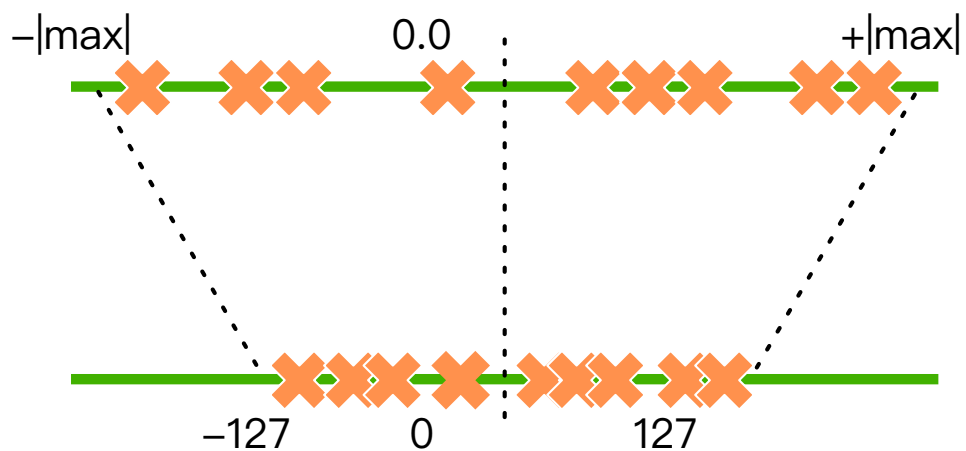
量化方法	功能	适用场景	使用条件	易用性	精度损失	预期收益
量化感知训练 (QAT)	通过finetune训练将模型量化误差降到最小	对量化敏感的场景, 模型, 例如目标检测, 分割等	有大量带标签的数据	好	极小	连续存续空间 4X,降低计算内存
静态离线量化 (PTQ (S))	通过少量校准数据得到量化模型	对量化不敏感的场景, 例如图像分类任务	有少量无标签数据	较好	较少	连续存续空间 4X,降低计算内存
动态离线量化 (PTQ(d))	进量化模型的科学系权重	模型开销大, 访存开销大的模型	无	一般	一般	连续存续空间 4X,降低计算内存

量化

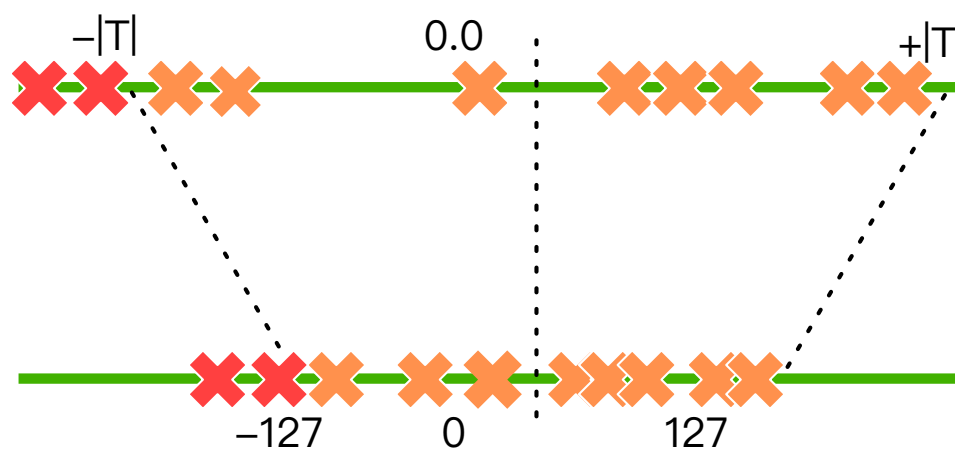
□ 量化原理

- 模型量化桥接了定点与浮点，建立了一种有效的数据映射关系，使得以较小的精度损失代价获得了较好的收益。

No Saturation: map $|\max|$ to 127



Saturation above $|\text{threshold}|$ to 127



量化

□ 量化原理

- 要弄懂模型量化的原理就是要弄懂这种数据映射关系，浮点与定点数据的转换公式如下：

$$Q = \frac{R}{S} + Z$$

$$R = (Q - Z) * S$$

- R 表示输入的浮点数据
- Q 表示量化之后的定点数据
- Z 表示零点（Zero Point）的数值
- S 表示缩放因子（Scale）的数值

量化

□ 量化原理

- 求解 S 和 Z 有很多种方法，这里列举中其中一种线性量化的求解方式（MinMax）如下：

$$S = \frac{R_{\max} - R_{\min}}{Q_{\max} - Q_{\min}}$$

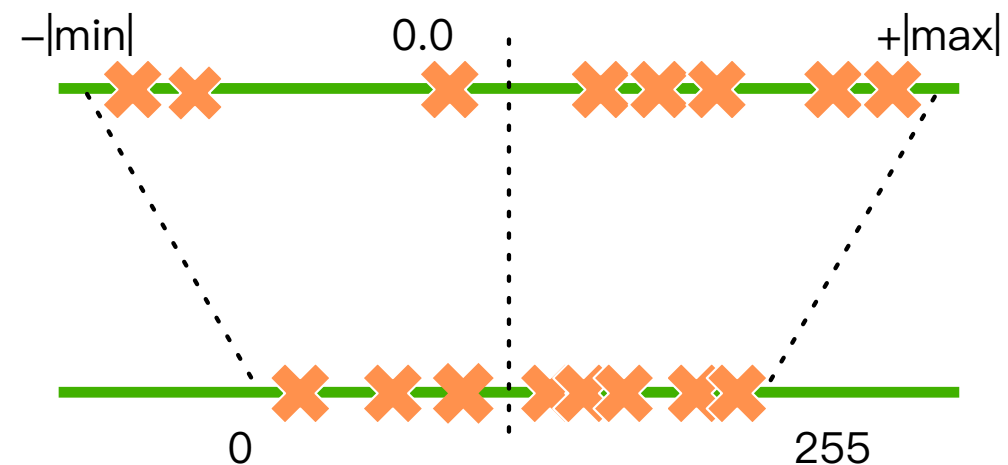
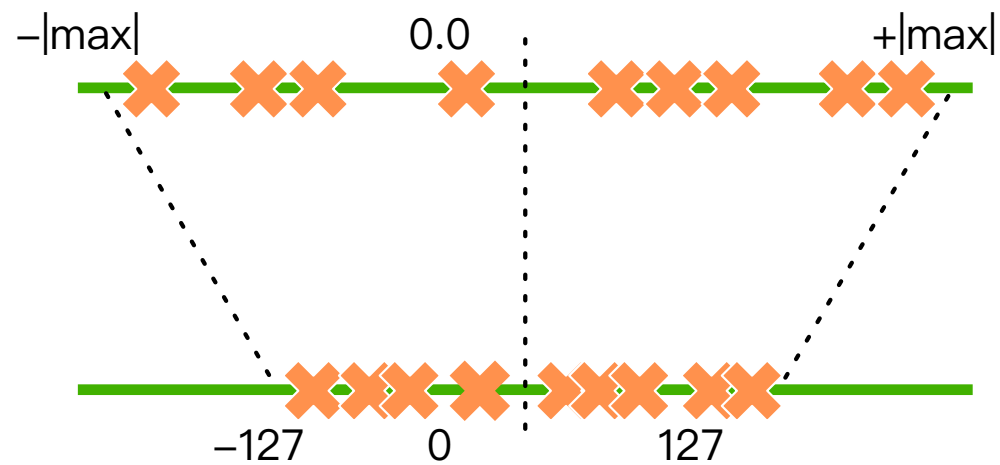
$$Z = Q_{\max} - R_{\max} \div S$$

- Rmax 表示输入浮点数据中的最大值
- Rmin 表示输入浮点数据中的最小值
- Qmax 表示最大的定点值 (127 / 255)
- Qmin 表示最小的定点值 (-128 / 0)

量化

□ 量化类型

- 对称量化
- 非对称量化



量化

□ 量化

- **GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformer (ICLR2023)**
([\[2210.17323\] GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers \(arxiv.org\)](#))
- **SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models (ICML 2023)**
([\[2211.10438\] SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models \(arxiv.org\)](#))
- **Training Transformers with 4-bit Integers (Neurips 2023)**
([\[2306.11987\] Training Transformers with 4-bit Integers \(arxiv.org\)](#))
- **Atom: Low-bit Quantization for Efficient and Accurate LLM Serving (arxiv 2023)**
([\[2310.19102\] Atom: Low-bit Quantization for Efficient and Accurate LLM Serving \(arxiv.org\)](#))

谢谢!